

Hochschule Bremerhaven

Fachbereich II
Management und Informationssysteme
Wirtschaftsinformatik B.Sc.

Modul
Vernetzte Systeme

ToDo-Liste

Webanwendung innerhalb eines vernetzten Systems

Vorgelegt von:	Mert Özdemir	MatNr. 41193
	Celal Akköprü	MatNr. 39652
	Mustafa Thamer	MatNr. 39628
	Mohammed Alzubaidy	MatNr. 40085
Vorgelegt am:	9. März 2025	
Dozent:in:	Prof. Dr. Oliver Radfelder und Prof. Dr. Karin Vosseberg	

Inhaltsverzeichnis

1	Einleitung	4
2	Projektorganisation und Teamarbeit	4
3	Erfahrungen und Erkenntnisse aus dem Modul Vernetzte Systeme	5
4	Technologische Grundlagen	7
4.1	Docker: Grundlagen und Anwendung	7
4.1.1	Der Zitierstil	8
4.1.2	Das Deckblatt	9
4.2	ISO/OSI-Modell im Kontext	12
5	Projektidee und Vorhaben	13
5.1	Float und FloatBarrier	17
5.2	Abbildungen	19
5.3	Listings	20
5.4	Tabellen	22
5.5	Labels und Verweise	26
6	Systemarchitektur	27
7	Implementierung und Ergebnisse	30
7.1	MariaDB Stresstest	31
7.2	Redis Benchmark	32
7.3	HAProxy Session-Stickiness	33
7.4	HAProxy Load-Balancing Test	34
7.4.1	Erweiterter Load-Balancing Test unter erhöhter Last	35
7.5	Umfassender Netzwerk- und Performance-Test	35
7.6	TCP Dump – Netzwerk- und Performance-Test	36
7.6.1	Erweiterter Netzwerk- und Performance-Test	37
7.7	k6 Lasttest – Performance-Analyse von HAProxy und Apache	38
7.7.1	k6 Login-Lasttest	39
7.7.2	k6 Add-ToDo-Liste	40
7.8	Diagramm – Gnuplot: CPU- und RAM-Nutzungsanalyse	41
8	Fazit und Ausblick	42
	Literaturverzeichnis	43
	Abbildungsverzeichnis	44

Tabellenverzeichnis	45
Listingverzeichnis	46
Anhang	47
I Die Poster	48
II Beamer	49
III Quickbuild	52
Selbstständigkeitserklärung	53

1 Einleitung

Cloud-Dienste, Datenbanken oder das Internet – all diese Technologien basieren auf der Kommunikation zwischen verschiedenen Systemen. Die Interaktion erfolgt durch eine Client-Host-Architektur, die eine schnelle, stabile und effiziente Datenübertragung sowie die reibungslose Verarbeitung von Anfragen ermöglicht. In der modernen IT-Welt entfernen sich Systeme zunehmend von klassischen Serverräumen und entwickeln sich hin zu vernetzten Server- und Netzwerkarchitekturen.

Die Bedeutung von verteilten und vernetzten Systemen nimmt stetig zu, da sie auf Netzwerken basieren, die die Grundlage der heutigen IT-Infrastruktur bilden. Der Begriff verteilte Systeme bezieht sich in diesem Zusammenhang auf eine Sammlung unabhängiger Einheiten, die miteinander kommunizieren, um ein größeres und effizienteres Gesamtsystem zu bilden. Dies lässt sich mit einer Firma mit verschiedenen Abteilungen vergleichen: Jede Abteilung arbeitet eigenständig, aber durch den kontinuierlichen Austausch von Informationen entsteht eine funktionierende Gesamtstruktur.

Diese Konzepte bilden auch die Grundlage des Moduls "Vernetzte Systeme"(VNS), in dem wir die Möglichkeit hatten, uns intensiv mit der Gestaltung, Implementierung und Verwaltung verteilter Systeme auseinanderzusetzen. Ein zentraler Bestandteil dieser Veranstaltung ist der Einsatz von Docker-Umgebungen, die es ermöglichen, komplexe Anwendungen effizient in Containern zu organisieren und zu betreiben.

Da Teamarbeit eine essenzielle Rolle in der modernen IT-Welt spielt, haben wir uns im Rahmen des VNS-Projekts zu einem Team zusammengeschlossen, um die erlernten Konzepte und Techniken in einem praxisnahen Projekt anzuwenden. Durch diesen praktischen Ansatz möchten wir nicht nur innovative Lösungen entwickeln und die Herausforderungen verteilter Systeme bewältigen, sondern auch unsere Problemlösungsfähigkeiten und Zusammenarbeit im Team weiter verbessern.

2 Projektorganisation und Teamarbeit

Für den Erfolg eines Projekts sind eine gute Organisation und eine effektive Zusammenarbeit im Team unerlässlich. In unserem Projekt im Modul "Vernetzte Systeme" haben wir, nachdem wir die Projektidee (To-do List) festgelegt hatten, die Aufgaben klar untereinander aufgeteilt. Jedes Teammitglied hatte eine bestimmte Verantwortung, was uns half, parallel zu arbeiten und das Projekt effizient und zügig umzusetzen. Wir haben ein flexibles Arbeitsmodell genutzt, das sowohl Online- als auch Präsenzmeetings umfasste, um eine kontinuierliche Kommunikation und einen reibungslosen Arbeitsablauf zu gewährleisten. Alle zwei Tage haben wir uns über Jitsi getroffen, um den Fortschritt zu besprechen, Ideen auszutauschen und Probleme

gemeinsam zu lösen. Diese regelmäßigen und kurzen Meetings halfen uns, fokussiert zu bleiben und Verzögerungen zu vermeiden. Zusätzlich dazu haben wir uns alle zwei Wochen in der Universität getroffen, um komplexere Themen zu diskutieren, den Code gemeinsam zu überprüfen und konkrete Pläne für die nächsten Schritte zu erstellen. Durch diese klare Struktur und regelmäßigen Meetings konnten wir das Projekt effizient verwalten und sicherstellen, dass alle Teammitglieder aktiv beteiligt waren. Die Kombination aus flexiblen Online-Meetings und persönlichen Treffen ermöglichte uns eine perfekte Balance zwischen Produktivität und Zusammenarbeit. Außerdem haben wir unseren Fortschritt kontinuierlich dokumentiert, was es uns erleichterte, den Überblick über erledigte und noch offene Aufgaben zu behalten. Am Ende half uns dieser strukturierte Ansatz, das Projekt erfolgreich und ohne größere Hindernisse abzuschließen.

3 Erfahrungen und Erkenntnisse aus dem Modul Vernetzte Systeme

Wenn ihr das Template herunterladet und entpackt (dazu später mehr), dann liegt ein fertiges und kompilierbares $\text{\LaTeX}$ Projekt vor. Genau genommen liegt dieses Dokument inkl. aller Inhalte vor, lediglich das Deckblatt unterscheidet sich etwas von der Version, die ich euch zum Download anbiete. Ihr könnt also im Detail nochmal nachschauen, wie bestimmte Beispiele in $\text{\LaTeX}$ aussehen. Wenn das tar entpackt wird, liegt folgende Verzeichnisstruktur vor:

```
/
├── beispiele.tex
├── bibtex/
│   └── hauptdatei.bib
├── buildlualatex.sh
├── hauptdatei.pdf
├── hauptdatei.tex
├── log/
├── src
│   ├── abbildungen
│   │   └── logoneu.png
│   ├── anhang/
│   │   ├── abbildungen/
│   │   ├── anhang.tex
│   │   └── anhang_deckblatt.tex
│   ├── basic_structure/
│   │   ├── abkuerzungen.tex
│   │   ├── deckblatt.tex
│   │   ├── erklaerung.tex
│   │   └── trennung.tex
│   ├── einleitung.tex
│   ├── fazit.tex
│   ├── hauptabschnitt_3.tex
│   └── struktur.tex
```

Abbildung 3.1: Verzeichnisstruktur

In der `beispiele.tex` finden sich kleine snippets zu Tabellen, Listings, Abbildungen usw. Seht es als kleinen Spickzettel für den Alltag, für unser Projekt hat diese Datei erstmal keine Bedeutung.

Das `bibtex` Verzeichnis und die darin enthaltende Datei `hauptdatei.bib` sind Teil der Literaturverwaltung von $\text{\LaTeX}$, die wir uns später noch anschauen werden.

Das `buildlualatex.sh` Skript enthält den Programmaufruf zum Kompilieren unseres Projektes. Dieses Skript könnt ihr unverändert lassen, es enthält bereits alle notwendigen Befehle und kann zum Bauen einfach mittels `./buildlualatex.sh` ausgeführt werden.

Die `hauptdatei.pdf` ist das Resultat unseres Bauprozesses, dieses PDF lest ihr gerade. In der `hauptdatei.tex` werden alle Pakete eingebunden, das Grundlayout definiert, die Verzeichnisse gebaut und vor allem der eigentliche Inhalt eingebunden. Diese Datei werden wir uns im nächsten Abschnitt ausführlich anschauen.

Das `log` Verzeichnis enthält alle Log- und Metadateien, die beim Bauprozess erstellt werden, auch hiermit müssen wir uns nicht allzu viel beschäftigen.

Das `src` Verzeichnis enthält den eigentlichen Inhalt unseres PDFs. Es ist strukturiert durch die Verzeichnisse `abbildungen`, für alle Grafiken, welche wir nutzen wollen, `anhang` für alle Inhalte und Abbildungen des Anhangs, `basic_structure` für das Einbinden von Standarddateien wie Deckblatt, Selbstständigkeitserklärung, Trennvorschläge usw. und die `.tex` Dateien in welche unser eigentlicher Inhalt kommt. Dieser Text steht bspw. in der Datei `struktur.tex`.

Wenn ihr in diesem Projekt arbeitet, werdet ihr zum größten Teil eure eigenen `.tex` Dateien und Abbildungen in `src` hinterlegen. Damit diese auch eingebunden und die korrekten Daten auf dem Deckblatt hinterlegt werden, widmen wir uns aber zunächst der `hauptdatei.tex`.

4 Technologische Grundlagen

Die `hauptdatei.tex` gliedert sich grob in vier Abschnitte: die Definition der Pakete, des Deckblattes und Zitierstils sowie die eigentliche Einbindung des Inhaltes. Alle Bereiche wollen wir uns im Folgenden einmal anschauen.

4.1 Docker: Grundlagen und Anwendung

Fast alle Funktionen und Features sind in $\text{\LaTeX}$ in sogenannten `packages` enthalten, welche ähnlich wie in Java importiert bzw. eingebunden werden müssen. In der Präambel werden all diese Pakete definiert. Zusätzlich wird in der Präambel die allgemeine Dokumentenklasse festgelegt. Also ob wir gerade einen Brief, ein Buch oder einen Artikel schreiben. Die Präambel beginnt dabei mit der `\documentclass` und endet mit Beginn des eigentlichen Dokumentes, also `\begin{document}`. Die sog. `documentclass` sieht für dieses Dokument folgendermaßen aus:

```
1 \documentclass[a4paper,12pt,parskip=half,headsepline,DIV=12,numbers=noenddot]{scrartcl}
   \begin{document}
```

Listing 4.1: Die `documentclass`

Wie wir sehen, wird in der `documentclass` z. B. das Format (A4) und die Schriftgröße (12pt.) definiert. Ebenso legen wir mit `scrartcl` fest, dass wir einen Artikel schreiben. Es handelt sich dabei um eine sog. KOMA-Script Klasse. Diese Klassen sind in der $\text{\LaTeX}$ Welt im Prinzip Standard und ein Blick in die [Dokumentation](#) lohnt sich allemal. Ebenso werden nicht, wie aus Word bekannt, Seitenränder festgelegt, sondern Anhand des sog. Satzspiegels eine optimale Verteilung von Text zu Seitengröße ermittelt. Hierfür dient die Angabe `DIV=12`, auch hier empfiehlt sich die Dokumentation der KOMA-Script Klassen.

Zu Beginn sollte die `documentclass` unverändert bleiben, kann jedoch mit einiger Erfahrung an die eigenen Bedürfnisse angepasst werden. Auf die `documentclass` folgt das Einbinden von Packages durch den Befehl `\usepackage`. Für die Darstellung der Verzeichnisstruktur in Abbildung 3.1 wird bspw. das Package `dirtree` genutzt. Einige Packages sind in der `hauptdatei.tex` mit kurzen Kommentaren versehen. Auch zu den Packages finden sich [Dokumentation](#), welche immer wieder zurate gezogen werden können und sollten. Neben der Einbindung der Packages können diese auch direkt definiert werden. So wird z. B. das Package `minted` für die Darstellung von Quellcode (siehe z. B. Listing 4.1) genutzt und das Layout direkt in der Präambel festgelegt:

```
1 \usepackage[outputdir=log, newfloat]{minted}
2 \setminted{
3     frame=lines,
4     framesep=2mm,
5     baselinestretch=1.2,
6     bgcolor=LightGray,
7     fontsize=\footnotesize,
8     linenos,
9     breaklines=true,
10    breakanywhere=true,
11    autogobble,
12    tabsize=2
13 }
```

Listing 4.2: Package Definition

Ich werde an dieser Stelle nicht auf einzelne Packages und Einstellungen eingehen, empfehle für einen genauen Einblick aber ausdrücklich die einzelnen Dokumentationen. Auch hier gilt, dass die üblicherweise gebrauchten Packages eingebunden und definiert sind und nur in Sonderfällen zusätzliche Packages gebraucht werden.

4.1.1 Der Zitierstil

Auch der Zitierstil wird innerhalb der Präambel definiert. Wir werden uns in einem späteren Kapitel ausführlicher mit Literaturverwaltung und Zitaten in $\text{\LaTeX}$ beschäftigen. Aktuell soll uns das Festlegen eines Zitierstiles ausreichen.

In der `hauptdatei.tex` sind bereits zwei Zitierstile definiert, es muss nur einer ausgewählt werden. Die Stile `ieee` und `apa` sind dabei in der Informatik gängig, entscheidet entsprechend eurem eigenen Geschmack und natürlich nach Vorgabe eurer Lehrenden.


```
1 % Zitierung nach IEEE
2
3 \usepackage[
4 backend=biber,
5 style=ieee,
6 autocite=inline,
7 ]{biblatex}
8 \addbibresource{bibtex/hauptdatei.bib}
9
10 % Zitierung nach APA
11
12 %\usepackage[
13 %backend=biber,
14 %style=apa,
15 %autocite=inline,
16 %]{biblatex}
17 %\addbibresource{bibtex/hauptdatei.bib}
```

Listing 4.3: Auswahl der Zitierstile

Um den Stil zu verändern, könnt ihr den jeweils anderen Stil mit einem % ein- bzw. auskommentieren. Aktuell ist der Stil `ieee` ausgewählt. Welche Unterschiede zwischen den Stilen bestehen, kann in den jeweiligen Guidelines nachgeschaut werden.

4.1.2 Das Deckblatt

Das Deckblatt ist unter `./src/basic_structure/deckblatt.tex` definiert. In vielen Fällen können die nötigen Parameter jedoch direkt in der Präambel festgelegt werden.

```
1 % true für Bachelorarbeit / false für Hausarbeit
2 \newbool{bachelorarbeit}
3 \setbool{bachelorarbeit}{true}
4
5 % Setze Fachbereich
6 \newcommand{\department}{Fachbereich II \ Management und Informationssysteme}
7
8 % Setze Studiengang
9 \newcommand{\studyprogram}{Wirtschaftsinformatik B.Sc.}
10
```

```

11 % Setze Modulname (bachelorarbeit muss false sein)
12 \newcommand{\modulname}{Modulname}
13
14 % Setze Dozent:in (bachelorarbeit muss false sein)
15 \newcommand{\auditor}{\textbf{Dozent:in:} \> Prof. Dr. Maxi Muster}
16
17 % Setze Gutachter:innen (bachelorarbeit muss true sein)
18 \newcommand{\firstauditor}{\textbf{Erstgutachter:} \> Prof. Dr. Maxi Mustermann}
19 \newcommand{\secondauditor}{\textbf{Zweitgutachterin:} \> Prof. Dr. Maxi Musterfrau}
20
21 % Setze Titel und Untertitel der Arbeit
22 \newcommand{\thetitle}{Eine unnötig komplizierte und sehr sehr sehr lange Abhandlung}
23 \newcommand{\thesubtitle}{Untersuchung einer Sache von wirklichem Wert}
24
25 % Setze Autor:in und MatNr.
26 \newcommand{\theauthor}{Maxi Mustermensch}
27 \newcommand{\matriculationnumber}{00000}
28
29 % Abstand zwischen Name und MatNr. (siehe Deckblatt)
30 \newcommand{\myspace}{1.0cm}
31
32 % Muss in src/basic_structure/deckblatt.tex einkommentiert werden!
33
34 \newcommand{\secondauthor}{\> Maxi Mustermensch \> MatNr. 00000\\}
35 \newcommand{\thirdauthor}{\> Maxi Mustermensch \> MatNr. 00000\\}
36 \newcommand{\fourthauthor}{\> Maxi Mustermensch \> MatNr. 00000\\}
37 \newcommand{\fifthauthor}{\> Maxi Mustermensch \> MatNr. 00000\\}

```

Listing 4.4: Definition des Deckblatts

Viele der Angaben sollten selbsterklärend sein und können sonst durch simples ausprobieren weiter erforscht werden. Beachtet jedoch, dass die Sonderzeichen `\>` und `\\` erhalten bleiben müssen, diese sind wichtig für das Layout. Ebenso sollte Zeile 30 bzw. die Angabe des `\myspace` bei mehreren Autor:innen beachtet werden. Der Wert steht für den Abstand zwischen Name und MatNr. und muss sich damit entsprechend am längsten Namen orientieren. Sollte eure Abgabe mehr als eine Autor:in haben, können die Werte für weitere Autor:innen direkt in der Präambel gesetzt werden, müssen jedoch zusätzlich in der `deckblatt.tex` einkommentiert werden.

Aus den in dem Listing 4.4 gewählten Einstellungen ergibt sich folgendes Deckblatt:

**Hochschule
Bremerhaven**

Fachbereich II
Management und Informationssysteme
Wirtschaftsinformatik B.Sc.

Bachelorarbeit
zur Erlangung des akademischen Grades
Bachelor of Science

**Eine unnötig komplizierte und sehr sehr sehr lange
Abhandlung**

Untersuchung einer Sache von wirklichem Wert

Vorgelegt von: Maxi Mustermensch MatNr. 00000

Vorgelegt am: 2. März 2023

Erstgutachter: Prof. Dr. Maxi Mustermann

Zweitgutachterin: Prof. Dr. Maxi Musterfrau

Abbildung 4.1: Deckblatt

Das Deckblatt ist manuell gesetzt und sollte nicht ohne entsprechendes Know-how angepasst werden. Natürlich könnt ihr immer etwas rumprobieren und euch zur Not das Projekt neu herunterladen.

4.2 ISO/OSI-Modell im Kontext

Wie bereits erwähnt, endet mit `\begin{document}` die Präambel, damit beginnt unser eigentlicher Inhalt. Der Befehl `\begin{document}` öffnet dabei eine Umgebung oder environment, welche durch `\end{document}` geschlossen wird. Dies ähnelt etwas dem `if` und `fi` aus der Bash und bildet einen Block.

Zu Beginn werden die Kopf- und Fußzeile des Dokuments beschrieben:

```

1 % Definition Header Sections sollen in der Kopfzeile stehen; Kopfzeile mit
   ↳ Unterstrich
2 \automark[section]{section}
3 \KOMAOPTIONS{headsepline=true}
4 %\ihead{Kopfzeile innen}
5 %\chead{Kopfzeile außen}
6 \ohead{\headmark}
7
8 % Definition footer \pagemark steht für Seitennummer
9 %\ifoot{Fußzeile innen}
10 %\cfoot{Fußzeile Mitte}
11 \ofoot{\pagemark}

```

Listing 4.5: Kopf- und Fußzeile

Daraus ergibt sich, wie in diesem Dokument zu sehen, eine Kopfzeile, welche die Überschrift (section) oder den Unterabschnitt (subsection) enthält und durch eine Linie abgegrenzt wird. Dabei wird immer nur die section bzw. subsection aufgeführt, welche als Erstes auf der Seite beginnt. Es existieren hier einige mögliche Definitionen, welche in der Dokumentation des Packages `scrlayer-scrpage` nachgelesen werden können. Die Fußzeile wird schlicht mit der aktuellen Seitennummer definiert.

Auf die Definition der Kopf- und Fußzeile folgt das Einbinden von Trennvorschlägen. $\text{\LaTeX}$ verfügt grundsätzlich über eine sehr gute automatische Silbentrennung, falls diese aber mal versagt, können eigene Trennvorschläge eingebunden werden. Beispiele hierzu sind in der `trennung.tex` hinterlegt.

Auf weitere kleinere Einstellungen und optionale Layoutvarianten, welche ausreichend kommentiert sind, folgt die Einbindung des Deckblattes, mit dem wir uns ja schon beschäftigt haben. Die nun folgenden inputs sorgen dafür, dass euer geschriebener Text auch in das Dokument eingebunden und angezeigt wird. So sorgen die folgenden Inputs dafür, dass der bisherige Text, den ihr in diesem Dokument lesen konntet, eingebunden wird.

```

1 \input{src/einleitung.tex}
2 \input{src/struktur.tex}
3 \input{src/zurhauptdatei.tex}

```

Listing 4.6: input statements

Auf die `inputs` folgen die jeweiligen Verzeichnisse, auch hier sollten die Einstellung nicht ohne entsprechende Kenntnisse geändert werden. Grundsätzlich werden die Verzeichnisse automatisch erstellt, lediglich für den Fall, dass ein Verzeichnis leer bleiben würde, da das Dokument z. B. keine Tabellen enthält, muss der entsprechende Teil der `hauptdatei.tex` auskommentiert werden. Dabei kann es passieren, dass sich die Kopfzeile nicht mehr korrekt verhält, da diese teilweise für die Verzeichnisse manuell angepasst werden muss.

Das Abkürzungsverzeichnis stellt eine Besonderheit dar, dieses muss manuell befüllt werden, dazu gibt es Beispiele in der `abkuerzungen.tex`.

Auf die Verzeichnisse folgt das Deckblatt für den Anhang und der Anhang selbst. Dort muss lediglich beachtet werden, dass für Überschriften auf höchster Ebene nur `subsections` verwendet werden dürfen, da der Anhang selbst die `section` darstellt.

Oftmals sieht man in $\text{\LaTeX}$, dass die Präambel in eine eigene Datei ausgelagert wird. Dies dient jedoch nur der Übersichtlichkeit und der etwas klareren Trennung, ich verzichte darauf in dieser Vorlage.

5 Projektidee und Vorhaben

Wie bereits erwähnt, wird Text in $\text{\LaTeX}$ als `plain-text` geschrieben. Was bedeutet, dass wir einfach Text schreiben können und dieser wird in der definierten Schriftart und -größe dargestellt. Zur Veranschaulichung soll uns folgendes kleines Textbeispiel dienen:

Er hörte leise Schritte hinter sich. Das bedeutete nichts Gutes. Wer würde ihm schon folgen, spät in der Nacht und dazu noch in dieser engen Gasse mitten im übel beleumundeten Hafenviertel? Gerade jetzt, wo er das Ding seines Lebens gedreht hatte und mit der Beute verschwinden wollte!

Der oben stehende Text sieht in der `.tex` Datei folgendermaßen aus:

```
1 Er hörte leise Schritte hinter sich.
2 Das bedeutete nichts Gutes.
3 Wer würde ihm schon folgen, spät in der Nacht
4 und dazu noch in dieser engen Gasse mitten
5 im übel beleumundeten Hafenviertel?
6 Gerade jetzt, wo er das Ding seines Lebens gedreht hatte
7 und mit der Beute verschwinden wollte!
```

Listing 5.1: Textbeispiel

Wir sehen, der Inhalt der beiden Ansichten ist identisch, das Layout jedoch nicht. In der Datei ist der Text auf sieben Zeilen verteilt, im fertigen Dokument wird allerdings nicht automatisch ein Umbruch eingefügt. Einen solchen Umbruch müssen wir manuell setzen:

Er hörte leise Schritte hinter sich.
Das bedeutete nichts Gutes.
Wer würde ihm schon folgen, spät in der Nacht
und dazu noch in dieser engen Gasse mitten
im übel beleumundeten Hafenviertel?
Gerade jetzt, wo er das Ding seines Lebens gedreht hatte
und mit der Beute verschwinden wollte!

In der Datei sieht dies folgendermaßen aus:

```
1 Er hörte leise Schritte hinter sich.\\
2 Das bedeutete nichts Gutes.\\
3 Wer würde ihm schon folgen, spät in der Nacht\\
4 und dazu noch in dieser engen Gasse mitten\\
5 im übel beleumundeten Hafenviertel?\\
6 Gerade jetzt, wo er das Ding seines Lebens gedreht hatte\\
7 und mit der Beute verschwinden wollte!\\
```

Listing 5.2: Textbeispiel 2

Einen Zeilenumbruch wird in $\text{\LaTeX}$ somit folgendermaßen realisiert: `\\`

Soll statt einem Umbruch eine Leerzeile folgen, muss auch in der Datei eine Leerzeile eingefügt werden. Wie in diesem Dokument schon mehrfach gesehen und kurz in der Einleitung erwähnt, können Eigennamen oder Wörter, die man hervorheben will, mit dem Befehl `\texttt{Beispieltext}` realisiert werden, dies sieht dann so aus: `Beispieltext`. Um Text fettgedruckt darzustellen, können wir das Kommando `\textbf{Beispieltext}` nutzen, dies sieht dann so aus: **Beispieltext**.

Wenn Text erst einmal unformatiert ausgegeben wird, dann müssen wir uns auch Gedanken machen, wie wir Überschriften darstellen können. Überschriften werden grundsätzlich mittels des Befehls `\section{Überschrift}` realisiert.

5 Überschrift

Abbildung 5.1: Beispiel für eine section

Mittels des Befehls `\subsection{Unterabschnitt}` kann ein Unterabschnitt erstellt werden. Für weitere Abstufungen kommt entsprechend der Befehl `\subsubsection{}` zum Einsatz.

5.1 Unterabschnitt

Abbildung 5.2: Beispiel für eine subsection

Die Grundlagen für das Textlayout sind damit gegeben. Es gibt noch einige zusätzliche Möglichkeiten z. B. das Einrücken von Text, wie es bei längeren Zitaten üblich ist:

Er hörte leise Schritte hinter sich. Das bedeutete nichts Gutes. Wer würde ihm schon folgen, spät in der Nacht und dazu noch in dieser engen Gasse mitten im übel beleumundeten Hafenviertel? Gerade jetzt, wo er das Ding seines Lebens gedreht hatte und mit der Beute verschwinden wollte!

Eine solche Einrückung ist über die `quote` environment möglich. Gleichzeitig möchte man oft eingerückten Text in einer etwas kleineren Schriftgröße darstellen. Dies bringt uns zu der Definition der Schriftgröße. Als Standard dient die in der `documentclass` festgelegte Schriftgröße, von der über Befehle nach oben oder unten abgewichen werden kann.

Tabelle 5.1: L^AT_EX Schriftgrößen

Befehl	Ergebnis
<code>\tiny</code>	winzig
<code>\scriptsize</code>	klein
<code>\footnotesize</code>	etwas größer
<code>\small</code>	noch etwas größer
<code>\normalsize</code>	normal
<code>\large</code>	groß
<code>\Large</code>	größer
<code>\LARGE</code>	noch größer
<code>\huge</code>	riesig
<code>\Huge</code>	noch riesiger

Damit sieht unser eingerückter Text als L^AT_EX Code folgendermaßen aus:

```

1 \begin{quote}
2   \footnotesize{
3     Er hörte leise Schritte hinter sich.
4     Das bedeutete nichts Gutes.
```

```

5   Wer würde ihm schon folgen, spät in der Nacht
6   und dazu noch in dieser engen Gasse mitten
7   im übel beleumundeten Hafenviertel?
8   Gerade jetzt, wo er das Ding seines Lebens gedreht hatte
9   und mit der Beute verschwinden wollte!}\
10 \end{quote}

```

Listing 5.3: Textbeispiel 3

Ebenso existieren einige Sonderzeichen, welche nur durch spezielle Befehle dargestellt werden können.

Tabelle 5.2: L^AT_EX Sonderzeichen

Befehl	Ergebnis
<code>\&</code>	<code>&</code>
<code>\%</code>	<code>%</code>
<code>\\$</code>	<code>\$</code>
<code>\#</code>	<code>#</code>
<code>\{</code>	<code>{</code>
<code>\}</code>	<code>}</code>
<code>\textasciitilde</code>	<code>~</code>
<code>\textasciicircum</code>	<code>^</code>
<code>\textbackslash</code>	<code>\</code>
<code>\glqq{ }</code>	<code>„</code>
<code>\grqq{ }</code>	<code>“</code>

Grund ist, dass alle diese Zeichen in L^AT_EX eine eigene Bedeutung haben. Entsprechend muss unterschieden werden können, ob wir einen L^AT_EX Befehl ausführen oder das Sonderzeichen im Text ausgeben wollen. Auf eines der Zeichen, die sog. Tilde (~) soll kurz eingegangen werden. Diese sieht man meistens vor oder nach Befehlen im Code, so z. B. jedes Mal, wenn der L^AT_EX (`\LaTeX~`) Schriftzug genutzt wird. Die Tilde steht dabei für ein Leerzeichen, welches wir in solchen Fällen explizit angeben müssen, da L^AT_EX nicht unterscheiden kann zwischen einem Leerzeichen zum Abgrenzen von Fließtext zum Kommando oder einem zu druckenden Leerzeichen. Wenn ihr also bemerkt, dass euch ein Leerzeichen zwischen einer Kommandoausgabe und Fließtext fehlt, nutzt die ~.

Die letzten beiden Kommandos die wir uns zur Formatierung von Fließtext angucken werden,

sind `\vspace*{}` und `\hspace*{}`. Das `\vspace*{1.0cm}` Kommando sorgt für vertikalen Weißraum von einem Zentimeter im Dokument:

Beispieltext

Beispieltext

Der `\hspace{}` steht für einen horizontalen Weißraum und wird von mir deutlich seltener genutzt. Ab und an wird dies aber gebraucht z. B. für den Abstand von Namen und MatrNr. auf dem Deckblatt. Zwischen diesen beiden Wörtern habe ich einen `\hspace{1.0cm}` gesetzt:

Beispieltext Beispieltext

Mittels all dieser Kommandos ist es uns nun möglich, Fließtext ansprechend zu formatieren. Unser Dokument besteht jedoch nicht ausschließlich aus Fließtext, sondern aus einer Mischung aus Text, Listings, Tabellen und Abbildungen. Aus diesem Grund werden wir uns nun mit diesen drei Punkten beschäftigen.

5.1 Float und FloatBarrier

Oft werden Abbildungen und Tabellen in sog. Floatumgebungen genutzt. Dabei übernimmt $\text{\LaTeX}$ die Aufgabe, die Elemente optimal im Fließtext zu positionieren und so einen angenehmen Lesefluss zu gewährleisten. Was in der Theorie sehr praktisch klingt, klappt m. E. in der Praxis nicht immer gut. Oftmals werden die Elemente um mehrere Seiten verschoben und passen so inhaltlich nicht mehr gut mit dem Geschriebenen zusammen. Ebenso verhindert die Floatumgebung, dass Tabellen über Seitengrenzen hinweg dargestellt werden können, dies ist nur logisch, aber oft nicht gewollt. Dennoch bietet die Verwendung auch Vorteile, so ist der Umgang mit captions (also der Abbildungsbeschreibung) oft einfacher und auch werden die Weißräume vor und nach den Inhalten sauber erstellt, sodass ein manuelles `\vspace` nicht nötig ist. In Abwägung dieser Punkte, habe ich mich dazu entschieden, weitestgehend auf Floats zu verzichten und im Zweifel etwas nachzukorrigieren. Da ihr Online aber oft auf diese Beispiele stoßen werdet und ich für klassische Abbildungen (Bilder) auch eine Floatumgebung verwende, soll kurz gezeigt werden, wie diese aussehen.

Für Floats wird immer eine Umgebung bzw. `environment` geöffnet, für Abbildungen sieht diese folgendermaßen aus:

```
1 \begin{figure}[!ht]
2   \includegraphics[width=0.8\textwidth]{abbildungen/Logo.png}
3   \caption[Beschreibung]{Beschreibung}
```

```

4 \label{Beschreibung}
5 \end{figure}

```

Listing 5.4: Floatumgebung für figures

Die Floatumgebung wird eingeleitet durch `\begin{figure}`, die Zeichenkombination in den eckigen Klammern steht dann für den sog. float specifier. Alle Werte haben dabei eine eigene Bedeutung und können entsprechend kombiniert werden.

Tabelle 5.3: float specifier

Specifier	Berechtigung
h	ungefähr an der gleichen Stelle, an der sie im Ausgangstext vorkommt (jedoch nicht genau an dieser Stelle)
t	Positionierung am oberen Rand der Seite
b	Am unteren Rand der Seite platzieren
p	Auf eine spezielle Seite
!	Überschreibt die internen Parameter, die LaTeX zur Bestimmung „guter“ Gleitpositionen verwendet.

Trotz der Angabe `!ht` ist nicht sichergestellt, dass die Abbildung auch wirklich an der gewünschten Stelle oder innerhalb eines gewünschten Bereiches erscheint. Um trotz Floatumgebung genauer festzulegen, in welchem Bereich die Abbildung erscheinen soll, kann das Kommando `\FloatBarrier` genutzt werden. Durch dieses Kommando wird der Bereich, in dem die Abbildung von $\text{\LaTeX}$ gesetzt wird, begrenzt. Werden die `\FloatBarrier` direkt ober- und unterhalb der Umgebung gesetzt, sollte die Abbildung an der exakten Stelle aus dem Code erscheinen. Wird zusätzlich noch Fließtext in den Bereich aufgenommen, wird die Abbildung innerhalb dieses Bereiches inkl. des Fließtextes optimal gesetzt.

```

1 \FloatBarrier
2 Er hörte leise Schritte hinter sich. Das bedeutete
3 nichts Gutes. Wer würde ihm schon folgen, spät
4 in der Nacht und dazu noch in dieser engen Gasse mitten
5 im übel beleumundeten Hafenviertel? Gerade jetzt, wo
6 er das Ding seines Lebens gedreht hatte und mit der
7 Beute verschwinden wollte!
8

```

```

9 \begin{figure}[!ht]
10   \includegraphics[width=0.8\textwidth]{abbildungen/Logo.png}
11   \caption[Beschreibung]{Beschreibung}
12   \label{Beschreibung}
13 \end{figure}
14 \FloatBarrier

```

Listing 5.5: Beispiel FloatBarrier

In diesem Beispiel kann die Abbildung nicht außerhalb der FloatBarrier gesetzt werden, jedoch zwischen den Text. So könnte ein Teil des Textes ober- und unterhalb der Abbildung stehen, die ganze Abbildung jedoch auch über oder unter dem Text dargestellt werden. Im zweiten Beispiel sollte die Abbildung an der exakten Stelle aus dem Code auch im fertigen Dokument erscheinen.

```

1 Er hörte leise Schritte hinter sich. Das bedeutete
2 nichts Gutes. Wer würde ihm schon folgen, spät
3 in der Nacht und dazu noch in dieser engen Gasse mitten
4 im übel beleumundeten Hafenviertel? Gerade jetzt, wo
5 er das Ding seines Lebens gedreht hatte und mit der
6 Beute verschwinden wollte!
7
8 \FloatBarrier
9 \begin{figure}[!ht]
10   \includegraphics[width=0.8\textwidth]{abbildungen/Logo.png}
11   \caption[Beschreibung]{Beschreibung}
12   \label{Beschreibung}
13 \end{figure}
14 \FloatBarrier

```

Listing 5.6: Beispiel 2 FloatBarrier

Auch für Tabellen existiert eine Floatumgebung, welche jedoch in diesem Template nicht genutzt wird. Die Floatumgebung für Tabellen wird mit `\begin{table}` eingeleitet und entsprechend beendet.

5.2 Abbildungen

Wenn wir Abbildungen, in diesem Fall speziell Bilder, einbinden, dann binden wir diese mit dem Kommando `\includegraphics` ein. Zusätzlich können bzw. müssen weitere Werte mitgegeben

werden. Zum einen, wo die einzubindende Datei liegt und zum anderen welche Größe das Bild erhalten soll. Wie beschrieben, kann die Abbildung dann noch in eine Floatumgebung eingebunden werden. Das einfachste Beispiel könnte folgendermaßen aussehen:

```
1 \includegraphics[width=0.8\textwidth]{abbildungen/Logo.png}
```

Listing 5.7: Beispiel für das Einbinden einer Abbildung

Der Wert in den eckigen Klammern beschreibt die Größe der Abbildung, in diesem Fall, hat die Abbildung den Faktor 0.8 der Textbreite. Setzen wir um das Kommando die Floatumgebung, können wir noch weitere Möglichkeiten nutzen:

```
1 \begin{figure}
2   \centering
3   \includegraphics[width=0.8\textwidth]{abbildungen/Logo.png}
4   \caption[Verzeichniseintrag]{Beschreibung}
5   \label{labelname}
6 \end{figure}
```

Listing 5.8: Beispiel 2 für das Einbinden einer Abbildung

Der Befehl `\centering` steht, wenig überraschend, für das horizontale Zentrieren der Abbildung. Das `\caption` Kommando ermöglicht das Setzen der Abbildungsbeschreibung, die Angabe innerhalb der eckigen Klammern steht dabei für den Eintrag im Abbildungsverzeichnis, der Wert in den geschweiften Klammern für die Abbildungsbeschreibung direkt unterhalb der Abbildung. In den allermeisten Fällen sollten die Angaben identisch sein, in solchen Fällen, kann die Angabe für den Verzeichniseintrag (inkl. der Klammern) einfach weggelassen werden.

5.3 Listings

Listings bzw. die Darstellung von Quellcode stellen in diesem Template eine kleine Besonderheit dar. In Dokumentationen oder Onlineforen findet man für Listings oft die `lstlisting` Umgebung, in diesem Template wird diese nicht genutzt, sondern das `minted` Paket. Dieses ermöglicht m. E. deutlich schönere Listings inkl. deutlich umfangreicherem Highlighting. Da `minted` jedoch nicht außerhalb von Floatumgebungen ohne weiteres mit Captions umgehen kann, habe ich mich entschieden selbst eine Umgebung zu schreiben. Diese Umgebung trägt den Namen `code` und kann so nur in diesem Template genutzt werden.

```

1 \begin{code}{Beschreibung}{label}
2   \begin{minted}{bash}
3     #!/bin/bash
4
5     number="$1"
6     if test $number -eq 0; then
7       echo "true"
8     else
9       echo "false"
10    fi
11  \end{minted}
12 \end{code}

```

Listing 5.9: Beispiel für ein Listing

Wie üblich wird die Umgebung mit dem Befehl `\begin{}` eingeleitet. In diesem Fall müssen wir die `code` Umgebung erstellen und innerhalb dieser dann die eigentliche `minted` Umgebung. Für die `code` Umgebung kann direkt die Listingbeschreibung gesetzt werden. In den geschweiften Klammern der `minted` Umgebung muss dann die Sprache z. B. Bash, Java oder Python gesetzt werden, daraufhin folgt der dazustellende Code. Statt den Quellcode innerhalb der `minted` Umgebung direkt zu schreiben, kann auch eine Datei eingebunden werden.

```

1 \inputminted{latex}{src/listingcodeexample.txt}

```

Listing 5.10: Beispiel 2 für ein Listing

Beachtet, dass `inputminted`, wie in Listing 5.9 zu sehen, innerhalb der `code` Umgebung stehen muss. Wollen wir nur einen Befehl oder kleinen Codeschnipsel zeigen, kann dieser auch direkt im Fließtext angezeigt werden. Dies könnte z. B. so aussehen:

Überschriften können in HTML mittels `\overschrift` dargestellt werden.

Im Code sieht dies dann folgendermaßen aus:

```

1 Überschriften können in HTML mittels \mintinline{html}|<h1>Überschrift</h1>|
  ↳ dargestellt werden.

```

Listing 5.11: Inline Listings

5.4 Tabellen

Tabellen sind gegenüber Abbildungen und Listings deutlich individueller und entsprechend existieren unzählige Möglichkeiten, Tabellen zu gestalten. Ich werde versuchen, die Grundlagen so zu erklären, dass ihr in den meisten Fällen ohne zusätzlichen Aufwand Tabellen erstellen könnt.

Für Tabellen stehen verschiedene Packages zur Verfügung. In diesem Template wird das `tabularray` Package genutzt. Dieses ermöglicht eine Vielzahl von Einstellungen, besitzt eine sehr einfache Einbindung der `booktabs` Bibliothek und ist sehr gut dokumentiert.

Die erwähnten `booktabs` beschreiben ein besonderes Tabellendesign, welches an Tabellen aus dem wissenschaftlichen Bereich angelehnt ist. Dabei wird auf vertikale und doppelte Linien verzichtet, ebenso müssen die Linien in Bezug auf Dicke und Länge nicht gesondert definiert werden.

Im Folgenden werden wir uns mit zwei Arten von Tabellen in $\text{\LaTeX}$ anschauen. Zum einen Tabellen, welche nicht mit der Seite umbrechen können und zum anderen Tabellen, welche über Seitengrenzen hinweg dargestellt werden können. Die beiden Umgebungen, die wir hierfür verwenden, sind `talltblr` und `longtblr`. Die `talltblr` Umgebung wird verwendet, um Tabellen auf einer Seite darzustellen und bietet gleichzeitig einen guten Umgang mit Captions und Labels.

```

1 \begin{center}
2   \begin{talltblr}[caption={Beispiel}, label={beispiel}]{colspec={X[c,m] X[c,m]},
   ↪   width=0.4\textwidth}\toprule
3     Überschrift 1 & Überschrift 2   \\\ \midrule
4     Wert 1.1   & Wert 1.2           \\\ \cmidrule{1-2}
5     Wert 2.1   & Wert 2.2           \\\ \cmidrule{1-2}
6     Wert 3.1   & Wert 3.2           \\\ \cmidrule{1-2}
7     Wert 4.1   & Wert 4.2           \\\ \cmidrule{1-2}
8     Wert 5.1   & Wert 5.2           \\\ \bottomrule
9   \end{talltblr}
10 \end{center}

```

Listing 5.12: Beispielcode für Tabelle

In diesem Beispiel wird zu Beginn eine `center` Umgebung eingeleitet, dies tue ich in fast allen Fällen, da Tabellen in den meisten Fällen zentriert dargestellt werden sollen, dennoch ist dies natürlich optional. Erst jetzt folgt die eigentliche Tabellenumgebung `talltblr`. Auf diese folgt in den eckigen Klammern die Definition der Caption und des Labels. In den geschweiften Klammern definieren wir nun das Layout der Spalten, an dieser Stelle beginnt es etwas kompliziert zu werden.

Die Angabe `colspec={X[c,m] X[c,m]}` definiert zwei Spalten, wobei Spalten vom Typ `X` für

Spalten mit fester Breite und automatischem Zeilenumbruch stehen. Der Wert `c` steht für eine vertikal und die Angabe `m` für eine horizontal zentrierte Ausrichtung der Spalte. Es existieren entsprechend Angaben für eine rechtsbündige (`r`) und eine linksbündige (`l`) Ausrichtung.

Aus dem Code aus Listing 5.12 ergibt sich folgende Tabelle:

Tabelle 5.4: Tabelle mit zwei Spalten

Überschrift 1	Überschrift 2
Wert 1.1	Wert 1.2
Wert 2.1	Wert 2.2
Wert 3.1	Wert 3.2
Wert 4.1	Wert 4.2
Wert 5.1	Wert 5.2

Die Angaben `\toprule`, `\midrule`, `\cmidrule` und `\bottomrule` stehen für die unterschiedlichen Arten von Linien innerhalb der Tabelle. Die zusätzliche Angabe von Ziffern bei der `\cmidrule` steht für die Anzahl der Spalten, die unterstrichen werden sollen. Der Spalteninhalt wird jeweils durch ein `&` abgegrenzt und jede Zeile mit einem Zeilenumbruch (`\\`) beendet.

Es können natürlich auch mehr als nur zwei Spalten definiert werden. Ebenso will man für die Überschriften des Öfteren ein leicht abweichendes Layout festlegen. Dies ist selbstverständlich möglich und kann folgendermaßen umgesetzt werden:

```

1 \begin{center}
2   \begin{tblr}[caption={Tabelle mit drei Spalten},
   ↪   label={beispielzwei}]{width=0.9\textwidth, colspec={X[l,m] X[c,m]
   ↪   X[l,m]}}\toprule
3   \textbf{Spalte 1} & \textbf{Spalte 2} & \SetCell[c=1]{c} \textbf{Spalte 3}
   ↪   \\ \midrule
4   a                & 1                & Ein ausreichend langer Text, damit es zu
   ↪   einem Zeilenumbruch kommt. \\ \cmidrule{1-3}
5   b                & 2                & Ein kürzerer Text.
   ↪   \\ \cmidrule{1-3}
6   c                & 3                & Wieder etwas längerer Text in dieser
   ↪   Spalte, auch hier kann es zu einem Zeilenumbruch kommen. \\ \cmidrule{1-3}
7   d                & 4                & Ein Text.
   ↪   \\ \cmidrule{1-3}
8   e                & 5                & Noch etwas zusätzlicher Text.
   ↪   \\ \bottomrule

```

```

9 \end{talltblr}
10 \end{center}

```

Listing 5.13: Beispiecode für Tabelle mit drei Spalten

In diesem Beispiel sind die Überschriften der jeweiligen Spalte zum einen mittels `\textbf{}` fett gedruckt worden, zum anderen ist die Überschrift der dritten Spalte nicht wie der übrige Inhalt linksbündig ausgerichtet, sondern zentriert. Hierfür wird das Kommando `\SetCell[c=1]{c}` genutzt. Die Angabe `c=1` steht für die Anzahl der folgenden Spalten (in diesem Fall eine) für die das in den geschweiften Klammern definiert Layout (in diesem Fall zentriert) gelten soll.

Die so beschriebene Tabelle wird dann folgendermaßen dargestellt:

Tabelle 5.5: Tabelle mit drei Spalten

Spalte 1	Spalte 2	Spalte 3
a	1	Ein ausreichend langer Text, damit es zu einem Zeilenumbruch kommt.
b	2	Ein kürzerer Text.
c	3	Wieder etwas längerer Text in dieser Spalte, auch hier kann es zu einem Zeilenumbruch kommen.
d	4	Ein Text.
e	5	Noch etwas zusätzlicher Text.

Es fällt auf, dass die ersten beiden Spalten eigentlich deutlich weniger Platz bräuchten, als sie einnehmen. Dies liegt daran, dass die zur Verfügung stehende Fläche unter allen Spalten gleich aufgeteilt wird. Dieses Verhalten können wir anpassen, indem wir den Spalten ein Größenverhältnis zueinander zuweisen:

```

1 \begin{center}
2 \begin{talltblr}[caption={Tabelle mit drei Spalten und angepasster Spaltenbreite},
  ↳ label={beispieldrei}]{width=0.9\textwidth, colspec={X[1,1,m] X[1,c,m]
  ↳ X[5,1,m]}}\toprule
3 \textbf{Spalte 1} & \textbf{Spalte 2} & \SetCell[c=1]{c} \textbf{Spalte 3}
  ↳ \\ \midrule

```



```

4      a                & 1                & Ein ausreichend langer Text, damit es zu
      ↪ einem Zeilenumbruch kommt.                \\ \cmidrule{1-3}
5      b                & 2                & Ein kürzerer Text.
      ↪ \\ \cmidrule{1-3}
6      c                & 3                & Wieder etwas längerer Text in dieser
      ↪ Spalte, auch hier kann es zu einem Zeilenumbruch kommen. \\ \cmidrule{1-3}
7      d                & 4                & Ein Text.
      ↪ \\ \cmidrule{1-3}
8      e                & 5                & Noch etwas zusätzlicher Text.
      ↪ \\ \bottomrule
9  \end{talltblr}
10 \end{center}

```

Listing 5.14: Beispiecode für Tabelle mit drei Spalten und angepasster Spaltenbreite

Die Zahlen innerhalb der Spalten Definition (`colspec={X[1,1,m] X[1,c,m] X[5,1,m]}`) sorgen nun für ein Größenverhältnis von eins zu fünf. Um die passenden Werte zu finden, braucht es ein bisschen Übung und natürlich auch ein paar Versuche. Unsere angepasste Tabelle hat nun eine angenehme und angemessene Verteilung:

Tabelle 5.6: Tabelle mit drei Spalten und angepasster Spaltenbreite

Spalte 1	Spalte 2	Spalte 3
a	1	Ein ausreichend langer Text, damit es zu einem Zeilenumbruch kommt.
b	2	Ein kürzerer Text.
c	3	Wieder etwas längerer Text in dieser Spalte, auch hier kann es zu einem Zeilenumbruch kommen.
d	4	Ein Text.
e	5	Noch etwas zusätzlicher Text.

Grundsätzlich können wir aus allen unseren Tabellen einfach seitenübergreifende Tabellen machen, indem wir statt `talltblr` einfach `longtblr` verwenden.

Wir können alle unsere Einstellungen beibehalten und müssen lediglich die Header unserer Tabelle festlegen. Dies ist nötig, damit die Überschriften auf der neuen Seite wiederholt werden können. So sieht die Tabelle 5.4 als `longtblr` fast genauso aus:

```

1 \begin{center}
2   \begin{longtblr}[caption={Tabelle mit zwei Spalten},
  ↪ label={beispieleins}]{colspec={X[c,m]
  ↪ X[c,m]},rowhead=1,width=0.4\textwidth}\toprule

```

```

3   Überschrift 1 & Überschrift 2 \\ \midrule
4   Wert 1.1      & Wert 1.2      \\ \cmidrule{1-2}
5   Wert 2.1      & Wert 2.2      \\ \cmidrule{1-2}
6   Wert 3.1      & Wert 3.2      \\ \cmidrule{1-2}
7   Wert 4.1      & Wert 4.2      \\ \cmidrule{1-2}
8   Wert 5.1      & Wert 5.2      \\ \bottomrule
9   \end{longtblr}
10  \end{center}

```

Listing 5.15: Beispiel für eine longtblr

Lediglich die Angabe `rowhead=1` steht als neue Angabe dafür, dass die erste Zeile die Header bzw. Überschriften enthält und somit bei jedem Seitenumbruch wiederholt werden soll.

Mittels dieser Angaben und etwas Übung sollten sich, ohne allzu viel Aufwand, ansehnliche und funktionale Tabellen erstellen lassen. Ich nutze oftmals direkt die `longtblr` Umgebung, dies ermöglicht aus meiner Sicht ein angenehmeres Layout.

5.5 Labels und Verweise

Wie in verschiedenen Listings gesehen und an einigen Textstellen erwähnt, existieren in $\text{\LaTeX}$ sog. labels. Durch das Labeln von Abbildungen, Tabellen, Listings und Überschriften können wir im Fließtext auf diese verweisen. Der Vorteil besteht darin, dass wir uns keine Gedanken um die Nummerierung von Abbildungen o. ä. machen müssen. Wird auf ein Label verwiesen, dann passt sich dieser Verweis automatisch an, sollte sich die Nummerierung geändert haben.

Wir können, in manchen Fällen müssen wir sogar, Labels für alle Überschriften, Tabellen, Listings und Abbildungen erstellen und somit auf diese referenzieren. Für Tabellen, Abbildungen und Listings habe ich die Labels in vielen Beispielen schon gezeigt, schaut euch hierfür nochmal die entsprechenden Listings aus den vorangegangenen Abschnitten an.

Überschriften können einfach mittels des Befehls `\label{}` zusätzlich mit einem Label versehen werden. So ist dieser Abschnitt folgendermaßen mit einem Label ausgestattet:

```

1  \subsection{Labels und Verweise}\label{labelundref}

```

Listing 5.16: Beispiel für ein Label

Um auf ein Label zu verweisen, z. B. Listing 5.16 kann der Befehl `\ref{}` genutzt werden. Innerhalb der geschweiften Klammern wird dann das passende Label hinterlegt, dies kann dann so aussehen:

1 Um auf ein Label zu verweisen, z. B. Listing~\ref{beispiellabel} kann der Befehl...

Listing 5.17: Beispiel für eine Referenz

6 Systemarchitektur

Wenn wir darüber sprechen, Hausarbeiten zu schreiben, dann brauchen wir in jedem Fall eine gute Literaturverwaltung. $\text{\LaTeX}$ bietet uns hierfür mit BibTeX ein sehr einfaches und verbreitetes Tool. Wir können es nutzen, um im Text zu zitieren bzw. zu paraphrasieren und um uns dann ein Literaturverzeichnis erstellen zu lassen. Genaugenommen arbeiten wir in diesem Template nicht mit BibTeX, sondern mit BibLaTeX, welches einen besseren Umgang mit Umlauten und Webseiten bietet. Ich verwende den Begriff BibTeX jedoch an dieser Stelle weiter.

Damit das Arbeiten mit BibTeX möglich ist, brauchen wir unsere Literatur als BibTeX Einträge in unserem Projekt. Ich habe hierfür ein Verzeichnis (bibtex) angelegt, welches eine Datei (hauptdatei.bib) enthält. In dieser Datei können wir unsere Literatureinträge hinterlegen. Glücklicherweise bietet quasi jede Bibliotheksseite einen entsprechenden Export in BibTeX an:

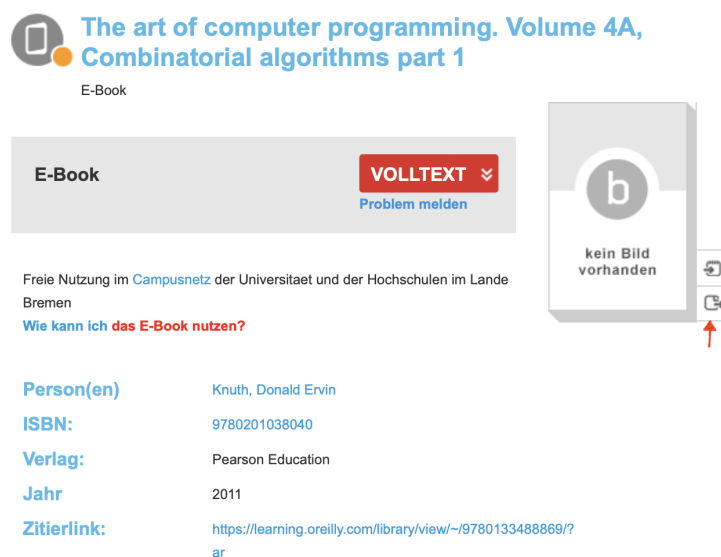


Abbildung 6.1: BibTeX Einträge exportieren

Die Möglichkeit sich zu Artikeln, Büchern, Sammelbänden etc. die passenden BibTeX Einträge generieren zu lassen, wird auf den entsprechenden Seiten quasi flächendeckend angeboten. Resultat eines solchen Exports ist ein einzelner BibTeX Eintrag, dieser kann dann in der hauptdatei.bib hinterlegt werden.



Abbildung 6.2: BibTeX Einträge exportieren (Formatauswahl)

```

1 @book{Thea2011,
2   title = {The art of computer programming. Volume 4A, Combinatorial algorithms part
3     ↪ 1},
4   author = {Donald Ervin Knuth},
5   publisher = {Pearson Education},
6   year = {2011},
7   note = {1 volume},
8   ISBN = {9780201038040},
9   URL = {https://learning.oreilly.com/library/view/~978013348869/?ar},
10  }

```

Listing 6.1: Beispiel BibTeX Eintrag

Im Laufe einer Hausarbeit wird unsere persönliche Bibliothek eine Vielzahl solcher Einträge enthalten. Die Einträge enthalten jeweils einen Typ, im obigen Beispiel book und ein Kürzel, in diesem Fall Thea2011. Es existieren viele verschiedene Typen von Einträgen, in den meisten Fällen werden wir unsere Einträge, wie gezeigt, exportieren können, es ist natürlich auch möglich, eigene Einträge zu erstellen. Hauptsächlich tue ich dies, wenn es sich um Webseiten oder ähnliche Onlinequellen handelt. Ein solcher Eintrag lässt sich gemäß nachstehendem Format erstellen:

```

1 @online{flignitz,
2   title = {How-to LaTeX},
3   url = {https://informatik.hs-bremerhaven.de/flignitz/latex.html},

```

```

4  urldate = {2023-04-18},
5  Author = {Fabian Lignitz},
6  Language = {de},
7  Month = {04},
8  Year = {2023},
9  }

```

Listing 6.2: Beispiel BibTeX Eintrag (Onlinequelle)

Soll eine Literaturangabe nach einem Zitat oder Paraphrase angefügt werden, ist dies durch das Kommando `\autocite[]{}{}` möglich. Zur Veranschaulichung sollen uns einige kleine Beispiele dienen:

„In general, each prime implicant corresponds in this way to a maximal subcube that stays within the set of points that make f true [1].“

When you hear about this problem for the first time, you might be tempted to ask a question of your own in return: „What? Are you serious that computer scientists still haven't figured out how to do such a simple thing?“ [1, S.432]

Gemäß Lignitz [2] biete BibLaTeX einen besseren Umgang mit Umlauten und Onlinequellen, als dies mit BibTeX möglich wäre.

Wie in den Beispielen zu sehen, kann an jeder beliebigen Stelle zitiert werden, zusätzlich kann in den eckigen Klammern eine Seitenzahl angegeben werden, dies ist jedoch nicht nötig. Zu beachten gilt, dass eingerückte Zitate keine Anführungsstriche erhalten, jedoch in diesem Fall in der Quelle Anführungsstriche gesetzt wurden und diese beibehalten werden müssen. Paraphrasen erhalten in keinem Fall Anführungszeichen. In $\text{\LaTeX}$ sehen die Beispiele wie folgt aus:

```

1  \glqq In general, each prime implicant corresponds in this way to a maximal
2  subcube that stays within the set of points that make f
3  true~\autocite[]{}{KnutThea2009}.\grqq{}
4
5  \begin{quote}
6    \footnotesize{
7      When you hear about this problem for the first time, you might be tempted
8      to ask a question of your own in return: \glqq What? Are you serious that computer
9      scientists still haven't figured out how to do such a simple
10     thing?\grqq{}~\autocite[S.432]{}{KnutThea2009}
11   }
12 \end{quote}
13

```

```

14 Gemäß Lignitz~\autocite[]{\flignitz} biete BibLaTeX einen besseren Umgang mit
15 Umlauten und Onlinequellen, als dies mit BibTeX möglich wäre.

```

Listing 6.3: Beispiele zitieren

Sobald wir in unserem Fließtext zitieren, wird entsprechend des gewählten Zitierstils auch das Literaturverzeichnis gebaut. Dies findet ihr am Ende des Hauptteils als erstes Verzeichnis. Durch die Verwendung von BibTeX brauchen wir uns also weder um das Anlegen von Literaturverzeichnis, noch um die Nummerierung oder Reihenfolge von Literaturangaben zu kümmern. Es reicht lediglich mittels des `~\autocite[]{}` Kommandos und des richtigen Kürzels im Fließtext eine Literaturangabe zu setzen.

7 Implementierung und Ergebnisse

In modernen vernetzten Systemen ist die Leistungsfähigkeit und Stabilität der Infrastruktur von entscheidender Bedeutung. Mit der zunehmenden Komplexität von Anwendungen und der steigenden Anzahl gleichzeitiger Nutzer wird es immer wichtiger, die Grenzen der Systeme zu verstehen und sicherzustellen, dass sie auch unter hoher Last zuverlässig funktionieren. Lasttests sind ein unverzichtbares Werkzeug, um die Performance, Skalierbarkeit und Stabilität von Systemen zu bewerten. Sie simulieren reale Belastungsszenarien und helfen, Engpässe, Schwachstellen und Optimierungspotenziale zu identifizieren.

In unserem Projekt haben wir umfangreiche Lasttests durchgeführt, um die Leistungsfähigkeit unserer Infrastruktur zu überprüfen. Dabei haben wir verschiedene Komponenten wie HAProxy, MariaDB und Redis unter die Lupe genommen. Mithilfe von Tools wie wrk, sysbench und k6 haben wir gezielt hohe Last erzeugt und die Reaktion des Systems analysiert. Die Ergebnisse dieser Tests liefern wertvolle Einblicke in das Verhalten unserer Anwendung unter realistischen Bedingungen und zeigen, wie gut das System mit einer großen Anzahl gleichzeitiger Anfragen umgehen kann.

Ein besonderer Fokus lag dabei auf der Datenbankperformance. MariaDB wurde mit intensiven Lese- und Schreiboperationen belastet, um zu überprüfen, wie effizient die Datenbank Anfragen verarbeitet und ob sie unter hoher Last stabil bleibt. Darüber hinaus haben wir die Lastverteilung durch HAProxy getestet, um sicherzustellen, dass Anfragen korrekt an die Backend-Server weitergeleitet werden und keine Engpässe entstehen. Auch die Performance von Redis als In-Memory-Cache wurde untersucht, um die Geschwindigkeit des Systems bei häufig abgerufenen Daten zu optimieren.

Die Ergebnisse unserer Lasttests zeigen, dass unser System insgesamt stabil und leistungsfähig ist. Allerdings gab es vereinzelt Latenzspitzen und Fehler, die auf Optimierungsbedarf hinweisen. Diese Erkenntnisse sind entscheidend, um die Infrastruktur weiter zu verbessern und sicherzustellen, dass sie auch in Zukunft zuverlässig arbeitet. In den folgenden Abschnitten

werden wir die Methodik, Durchführung und Ergebnisse der Lasttests detailliert darstellen und diskutieren.

7.1 MariaDB Stresstest

Um die Leistungsfähigkeit unserer MariaDB-Datenbank unter realistischen Bedingungen zu testen, haben wir einen umfangreichen Stresstest durchgeführt. Ziel war es, herauszufinden, wie gut die Datenbank mit einer hohen Anzahl gleichzeitiger Anfragen zurechtkommt. Dafür haben wir das Tool Sysbench verwendet, das sich hervorragend eignet, um die Performance von Datenbanken unter Last zu messen.

Zunächst haben wir sichergestellt, dass Sysbench auf unserem Work-Container installiert ist. Anschließend haben wir 15 Tabellen mit jeweils 20.000 Datensätzen erstellt, um eine realistische Auslastung zu simulieren. Mit diesem Setup haben wir zwei Haupttests durchgeführt: einen Lese-Last-Test und einen Schreib-Last-Test.

Beim Lese-Last-Test haben wir untersucht, wie schnell die Datenbank auf eine große Anzahl gleichzeitiger SELECT-Abfragen reagiert. Die Ergebnisse waren beeindruckend: Im Durchschnitt wurden 18.502 Abfragen pro Sekunde verarbeitet, was einer Rate von 1.156 Transaktionen pro Sekunde entspricht. Die durchschnittliche Latenz, also die Zeit, die eine Anfrage benötigt, lag bei 17,28 Millisekunden. Besonders erfreulich war, dass 95 % der Abfragen in weniger als 34,33 Millisekunden abgeschlossen wurden. Allerdings gab es vereinzelte Spitzenwerte, bei denen die Latenz auf bis zu 217,96 Millisekunden anstieg. Dennoch blieb die Datenbank stabil, und es gab keine Verbindungsabbrüche oder fehlerhaften Abfragen.

Der Schreib-Last-Test konzentrierte sich darauf, wie effizient die Datenbank INSERT- und UPDATE-Abfragen verarbeiten kann. Hier erreichten wir eine Rate von 13.631 Abfragen pro Sekunde bei 2.271 Transaktionen pro Sekunde. Die durchschnittliche Latenz lag bei 8,79 Millisekunden, und 95 % der Abfragen wurden in weniger als 17,95 Millisekunden abgeschlossen. Die maximale Latenz betrug hier 199,20 Millisekunden. Auch in diesem Test gab es nur eine einzige Fehlermeldung, die jedoch ignoriert werden konnte, da sie keinen Einfluss auf die Gesamtperformance hatte.

Was bedeuten diese Ergebnisse für uns? Insgesamt zeigt der Test, dass MariaDB auch unter hoher Last stabil und zuverlässig arbeitet. Die hohe Anzahl an Transaktionen pro Sekunde ist ein klarer Indikator dafür, dass die Datenbank effizient arbeitet. Allerdings haben wir festgestellt, dass es bei extrem hoher Last zu Latenzspitzen kommt, insbesondere beim Lese-Test. Diese Spitzen könnten darauf hinweisen, dass noch Optimierungspotenzial besteht.

Mögliche Ansatzpunkte für Verbesserungen sind die Optimierung von Indizes, um Daten schneller zu finden, die Implementierung von Query-Caching, um wiederholte Anfragen effizienter zu bearbeiten, und eine bessere Lastverteilung, um Engpässe zu vermeiden.

Zusammenfassend lässt sich sagen, dass MariaDB eine solide und leistungsfähige Datenbank ist, die auch unter hoher Belastung stabil läuft. Mit einigen gezielten Optimierungen könnten wir jedoch die Performance weiter verbessern und die Latenzspitzen reduzieren. Dieser Test hat uns wertvolle Einblicke geliefert, und wir sind zuversichtlich, dass wir die Datenbank noch effizienter machen können.

7.2 Redis Benchmark

Um die Leistungsfähigkeit unseres Redis-Servers zu testen, haben wir einen umfangreichen Benchmark durchgeführt. Ziel war es, herauszufinden, wie gut Redis mit einer hohen Anzahl gleichzeitiger Anfragen umgehen kann. Dafür haben wir das Redis-Benchmark-Tool verwendet, das speziell für solche Lasttests entwickelt wurde.

Der Test bestand aus zwei Hauptkomponenten: SET-Operationen, bei denen Daten in Redis gespeichert werden, und GET-Operationen, bei denen Daten aus Redis abgerufen werden. Um eine realistische Last zu simulieren, haben wir 100.000 Anfragen mit 50 parallelen Clients durchgeführt.

Die Ergebnisse im Detail

Bei den SET-Operationen, also dem Schreiben von Daten in Redis, wurden alle 100.000 Anfragen in nur 1,18 Sekunden abgeschlossen. Das entspricht einem beeindruckenden Durchsatz von 84.388 Anfragen pro Sekunde. Die durchschnittliche Latenz lag bei 0,320 Millisekunden, und 95 % aller Anfragen wurden in weniger als 0,471 Millisekunden abgeschlossen. Die maximale Latenz betrug 3,399 Millisekunden. Diese Ergebnisse zeigen, dass Redis Daten extrem schnell speichern kann, mit einer durchschnittlichen Verzögerung von weniger als einer halben Millisekunde.

Bei den GET-Operationen, also dem Abrufen von Daten aus Redis, wurden die 100.000 Anfragen in 1,45 Sekunden abgeschlossen. Der Durchsatz lag hier bei 69.204 Anfragen pro Sekunde, und die durchschnittliche Latenz betrug 0,377 Millisekunden. 95 % der Anfragen wurden in weniger als 0,599 Millisekunden abgeschlossen, während die maximale Latenz bei 11,087 Millisekunden lag. Auch hier zeigt sich, dass Redis unglaublich schnell ist, allerdings mit etwas höheren Latenzen als beim Schreiben.

Was bedeuten diese Ergebnisse für uns?

Die Ergebnisse des Benchmarks bestätigen, dass Redis eine extrem leistungsfähige In-Memory-Datenbank ist, die in der Lage ist, eine sehr hohe Anzahl von Anfragen in kürzester Zeit zu verarbeiten. Selbst bei 50 gleichzeitigen Clients bleiben die Antwortzeiten sehr niedrig, was Redis zu einer hervorragenden Wahl für Anwendungen macht, die schnellen Datenzugriff erfordern.

Allerdings haben wir festgestellt, dass die Latenz im GET-Test etwas höher ist als im SET-Test. Dies könnte verschiedene Ursachen haben. Zum einen könnten Cache-Misses eine Rolle spielen, bei denen einige Anfragen nicht direkt im Speicher gefunden werden und daher länger dauern. Zum anderen könnten Netzwerkverzögerungen oder eine höhere Systembelastung durch andere parallel laufende Prozesse zu den leicht erhöhten Latenzen beigetragen haben.

Insgesamt zeigt der Benchmark, dass Redis eine äußerst leistungsfähige und zuverlässige Lösung für Anwendungen ist, die schnellen Datenzugriff erfordern. Die Ergebnisse sind zufriedenstellend und selbst unter hoher Last bleibt Redis stabil und schnell. Mit einigen gezielten Optimierungen, wie der Reduzierung von Cache-Misses oder der Minimierung von Netzwerkverzögerungen, könnten wir die Performance jedoch noch weiter verbessern.

7.3 HAProxy Session-Stickiness

Um sicherzustellen, dass unsere Lastverteilung korrekt funktioniert und Benutzeranfragen während einer aktiven Session immer an denselben Server weitergeleitet werden, haben wir einen umfangreichen Session-Stickiness-Test mit HAProxy durchgeführt. Session Stickiness ist ein entscheidender Faktor, um eine konsistente Benutzererfahrung zu gewährleisten, da sie verhindert, dass Anfragen während einer Session zwischen verschiedenen Backend-Servern hin- und herspringen.

Der Test bestand aus mehreren Schritten, um die Funktionsweise von HAProxy unter die Lupe zu nehmen. Zunächst haben wir überprüft, ob Anfragen desselben Clients tatsächlich immer an denselben Backend-Server weitergeleitet werden. Dazu haben wir zehn aufeinanderfolgende Anfragen gesendet und analysiert, ob sie alle am selben Server ankamen. Zusätzlich haben wir die manuelle Cookie-Setzung getestet, indem wir explizit einen bestimmten Server (apache1) als Ziel vorgaben, um zu sehen, ob HAProxy diese Anweisung korrekt umsetzt.

Der Test begann mit einer ersten Anfrage, bei der das HAProxy-Session-Cookie gespeichert wurde. Dieses Cookie enthält die Information, welcher Backend-Server für die aktuelle Session zuständig ist. Anschließend haben wir zehn weitere Anfragen gesendet, um zu überprüfen, ob HAProxy diese Anfragen tatsächlich an denselben Server weiterleitet. In einem zweiten Schritt haben wir die manuelle Cookie-Setzung getestet, indem wir den Server `apache1` explizit als Ziel angaben. Dies diente dazu, die Flexibilität und Genauigkeit des HAProxy-Routings zu überprüfen.

Die Ergebnisse des Tests waren durchweg positiv. HAProxy leitete wiederholte Anfragen konsistent an denselben Server weiter, was bestätigt, dass die Session Stickiness korrekt funktioniert. Die Serverantworten zeigten, dass Apache als Backend-Server verwendet wurde, was die korrekte Funktionsweise der Stickiness unterstrich. Auch die manuelle Cookie-Setzung über `SSERVERID=apache1` funktionierte einwandfrei, und HAProxy leitete die Anfragen wie erwartet an den angegebenen Server weiter. Darüber hinaus blieben die Antwortzeiten stabil

und zeigten eine konsistente Geschwindigkeit, was darauf hindeutet, dass unser Load-Balancer effizient arbeitet und keine Engpässe verursacht.

Was bedeuten diese Ergebnisse für uns? Der Test hat gezeigt, dass unser HAProxy-Setup mit Session Stickiness wie erwartet funktioniert. Dies bedeutet, dass Benutzeranfragen während einer Session nicht zwischen den Backend-Servern hin- und hergeschoben werden, was eine konsistente Benutzererfahrung gewährleistet. Gleichzeitig funktioniert die Cookie-basierte Steuerung von HAProxy zuverlässig, sodass wir Anfragen gezielt an bestimmte Server leiten können. Die stabilen Antwortzeiten bestätigen, dass unser Load-Balancer effizient arbeitet und auch unter Last keine Performance-Einbußen verursacht.

Zusammenfassend lässt sich sagen, dass unser HAProxy-Setup mit Session Stickiness korrekt konfiguriert ist und zuverlässig arbeitet. Dies ist ein wichtiger Schritt, um eine konsistente und effiziente Benutzererfahrung zu gewährleisten. Mit diesem Setup können wir sicher sein, dass unsere Lastverteilung auch unter hoher Belastung stabil bleibt und unsere Anwendungen reibungslos funktionieren.

7.4 HAProxy Load-Balancing Test

Um die Leistungsfähigkeit und Stabilität unseres HAProxy-Setups zu überprüfen, haben wir einen umfangreichen Load-Balancing-Test durchgeführt. Für den Test haben wir das Tool wrk verwendet, das speziell für HTTP-Lasttests entwickelt wurde. Die Testbedingungen waren so gewählt, dass sie eine realistische Belastung simulieren: 4 Threads, 50 gleichzeitige Verbindungen und eine Testdauer von 10 Sekunden. Die getestete Instanz war unter der URL `http://haproxy:80` erreichbar.

Während des Tests wurden insgesamt 114.414 Anfragen erfolgreich verarbeitet. Die durchschnittliche Latenz lag bei 4,66 Millisekunden, was ein hervorragender Wert ist und die Effizienz unseres Systems unterstreicht. Die maximale Latenz betrug 62,49 Millisekunden, was ebenfalls noch im akzeptablen Bereich liegt. Pro Sekunde wurden im Schnitt 11.418,83 Anfragen bearbeitet, und die Übertragungsrate erreichte 21,03 MB/s. Es gab während des Tests 14 Lese-Fehler, was zwar nicht ideal ist, aber angesichts der Gesamtanzahl an Anfragen nur einen sehr geringen Anteil ausmacht. Verbindungsfehler, Schreibfehler oder Timeout-Probleme traten nicht auf.

Diese Ergebnisse zeigen, dass unser HAProxy-Setup stabil läuft und mit einer hohen Anzahl von Anfragen gut zurechtkommt. Die Performance ist insgesamt sehr stark und verlässlich, was bedeutet, dass wir mit unserer aktuellen Konfiguration auf einem soliden Stand sind. Dennoch sollten wir die Lese-Fehler im Auge behalten und gegebenenfalls Maßnahmen ergreifen, um diese weiter zu minimieren. Mögliche Ansätze hierfür könnten Optimierungen in der Netzwerkverbindung oder eine Feinjustierung der HAProxy-Konfiguration sein. Zusätzlich könnte es sinnvoll sein, in zukünftigen Tests die Anzahl der Threads und gleichzeitigen Verbindungen zu erhöhen, um zu sehen, wie unser System bei noch stärkerer Belastung reagiert.

7.4.1 Erweiterter Load-Balancing Test unter erhöhter Last

Um unser System noch stärker zu fordern, haben wir einen zweiten Load-Balancing-Test durchgeführt, bei dem wir die Testbedingungen deutlich verschärft haben. Diesmal haben wir die Anzahl der Threads auf 10 und die gleichzeitigen Verbindungen auf 150 erhöht. Der Test lief über einen Zeitraum von 30 Sekunden und wurde ebenfalls mit wrk durchgeführt.

In dieser Zeit wurden insgesamt 317.129 Anfragen verarbeitet und dabei 584,01 MB an Daten übertragen. Die durchschnittliche Latenz betrug 15,24 Millisekunden, wobei die höchste gemessene Latenz bei 249,78 Millisekunden lag. Die Anzahl der Anfragen pro Sekunde belief sich auf 10.922,29, was zeigt, dass unser System auch unter dieser deutlich höheren Last noch eine beeindruckende Performance liefern kann. Allerdings traten während des Tests 5.618 Lese-Fehler und 150 Timeouts auf. Obwohl dies im Vergleich zur Gesamtanzahl an Anfragen nur einen kleinen Prozentsatz ausmacht, deutet es darauf hin, dass das System unter dieser erhöhten Last an seine Grenzen stößt.

Für uns bedeutet das, dass unser HAProxy-Setup auch bei höherer Last grundsätzlich stabil läuft und eine gute Performance bietet. Die durchschnittliche Latenz von 15,24 ms ist zwar etwas höher als beim vorherigen Test, liegt aber noch im akzeptablen Bereich. Die gestiegene Anzahl an Lese-Fehlern und Timeouts zeigt jedoch, dass hier noch Optimierungspotenzial besteht. Mögliche Maßnahmen zur Verbesserung könnten eine weitere Feinjustierung der HAProxy-Konfiguration sein, z. B. durch das Anpassen von Timeouts oder die Erhöhung der maximalen gleichzeitigen Verbindungen. Auch ein genaueres Monitoring des Backend-Systems könnte helfen, Engpässe oder Flaschenhälse zu identifizieren.

7.5 Umfassender Netzwerk- und Performance-Test

Um die Interaktion verschiedener Komponenten unserer Infrastruktur zu analysieren, haben wir einen umfangreichen Netzwerk- und Performance-Test mit unserem Skript tcpdump-alles.sh durchgeführt. Dieses Skript kombiniert mehrere Tools wie TCPDump, wrk, sysbench und redis-benchmark, um die Aktivität und Stabilität unserer Infrastruktur unter realistischen Bedingungen zu prüfen.

Im ersten Schritt wurde die Netzwerkanalyse mit TCPDump gestartet, die sämtliche HTTP-, HTTPS-, MariaDB- und Redis-Anfragen erfasste. Parallel dazu wurde ein HTTP-Lasttest über HAProxy durchgeführt, bei dem wir mit curl und wrk mehrere GET-Anfragen an HAProxy gesendet haben, um den HTTP-Datenfluss zu simulieren. Danach folgte der MariaDB-Lasttest, bei dem mit sysbench eine intensive Schreib- und Lese-Belastung in der Datenbank erzeugt wurde. Anschließend wurde ein Redis-Benchmark durchgeführt, um die Performance von Redis beim Speichern und Abrufen von Daten zu testen. Abschließend erfolgten gezielte Tests mit Einzelzugriffen auf Redis, bei denen ein Key-Wert-Paar (*todos_cache*) gesetzt und ausgelesen wurde.

Während der gesamten Testphase wurden 1.283.518 Netzwerkpakete erfasst, von denen keine vom Kernel verworfen wurden, was auf eine saubere Paketverarbeitung hinweist. Bei der Analyse der erfassten Daten konnten wir feststellen, dass insgesamt 201.013 HTTP-Anfragen an Port 80 gerichtet wurden, was die hohe Anzahl der GET- und POST-Operationen während der HAProxy-Tests widerspiegelt. Bei den MariaDB-Operationen (Port 3306) wurden 580.952 Anfragen gezählt, was zeigt, dass die Datenbank während der Tests stark belastet wurde. Zusätzlich wurden 400.836 Redis-Operationen (Port 6379) registriert, was auf einen intensiven Cache-Einsatz hinweist.

Die HTTP-Statuscodes zeigten dabei überwiegend erfolgreiche Anfragen: 100.506 Anfragen erhielten den Statuscode 200 (OK), was auf eine fehlerfreie Verarbeitung hinweist. Allerdings gab es auch eine einzelne Anfrage mit dem Statuscode 400 (Bad Request), was auf einen fehlerhaften Request hinweisen könnte. Bei der detaillierten Fehleranalyse wurden 2 Fehler in den Logs erkannt, was in Anbetracht der hohen Gesamtanzahl an Anfragen minimal ist. Zusätzlich wurde 1 Timeout festgestellt, was darauf hindeutet, dass es möglicherweise kurzzeitig zu Verzögerungen in der Antwortzeit gekommen ist.

Insgesamt zeigt der Test, dass unsere Infrastruktur stabil und leistungsfähig ist. Die hohe Anzahl an erfolgreich verarbeiteten Anfragen und der geringe Anteil an Fehlern sprechen für eine gut konfigurierte Umgebung. Die wenigen Fehler und das eine Timeout deuten darauf hin, dass unter sehr hoher Last gelegentlich kleine Verzögerungen auftreten können. Hier könnten Optimierungen an den Timeouts in HAProxy, MariaDB oder Redis vorgenommen werden, um das System noch robuster zu machen. Auch ein genaueres Monitoring der Netzwerkverbindungen könnte sinnvoll sein, um mögliche Flaschenhälse zu erkennen und zu beheben. Insgesamt sind wir mit den Ergebnissen sehr zufrieden, da unser System die umfangreiche Belastung gut bewältigt hat und zuverlässig auf alle Anfragen reagierte. Die Tests haben uns wertvolle Einblicke geliefert, und wir sind zuversichtlich, dass wir unsere Infrastruktur weiter optimieren können, um zukünftige Lastspitzen noch besser zu bewältigen.

7.6 TCP Dump – Netzwerk- und Performance-Test

Um die Aktivität und Stabilität unserer Infrastruktur unter realistischen Bedingungen zu analysieren, haben wir einen umfangreichen Netzwerk- und Performance-Test mit unserem Skript `tcpdump-alles.sh` durchgeführt. Dieses Skript kombiniert mehrere Tools wie `TCPDump`, `wrk`, `sysbench` und `redis-benchmark`, um die Interaktion verschiedener Komponenten wie HAProxy, MariaDB und Redis zu prüfen.

Der Test begann mit einer Netzwerkanalyse mittels `TCPDump`, die sämtliche HTTP-, HTTPS-, MariaDB- und Redis-Anfragen erfasste. Parallel dazu wurde ein HTTP-Lasttest über HAProxy durchgeführt, bei dem wir mit `curl` und `wrk` mehrere GET-Anfragen an HAProxy gesendet haben, um den HTTP-Datenfluss zu simulieren. Anschließend folgte ein MariaDB-Lasttest, bei dem mit `sysbench` eine intensive Schreib- und Lese-Belastung in der Datenbank erzeugt wurde. Danach

wurde ein Redis-Benchmark durchgeführt, um die Performance von Redis beim Speichern und Abrufen von Daten zu testen. Abschließend erfolgten gezielte Tests mit Einzelzugriffen auf Redis, bei denen ein Key-Wert-Paar (*todos_{cache}gesetz**tundausgelesen**wurde*).

Während der gesamten Testphase wurden 1.283.518 Netzwerkpakete erfasst, von denen keine vom Kernel verworfen wurden. Dies deutet auf eine saubere Paketverarbeitung und stabile Netzwerkbedingungen hin. Bei der Analyse der erfassten Daten zeigte sich, dass insgesamt 201.013 HTTP-Anfragen an Port 80 gerichtet wurden, was die hohe Anzahl der GET- und POST-Operationen während der HAProxy-Tests widerspiegelt. Bei den MariaDB-Operationen (Port 3306) wurden 580.952 Anfragen gezählt, was zeigt, dass die Datenbank während der Tests stark belastet wurde. Zusätzlich wurden 400.836 Redis-Operationen (Port 6379) registriert, was auf einen intensiven Cache-Einsatz hinweist.

Die HTTP-Statuscodes zeigten überwiegend erfolgreiche Anfragen: 100.506 Anfragen erhielten den Statuscode 200 (OK), was auf eine fehlerfreie Verarbeitung hinweist. Allerdings gab es auch eine einzelne Anfrage mit dem Statuscode 400 (Bad Request), was möglicherweise auf einen fehlerhaften Request hindeutet. Bei der detaillierten Fehleranalyse wurden 2 Fehler in den Logs erkannt, was in Anbetracht der hohen Gesamtanzahl an Anfragen minimal ist. Zusätzlich wurde 1 Timeout festgestellt, was darauf hindeutet, dass es möglicherweise kurzzeitig zu Verzögerungen in der Antwortzeit gekommen ist.

7.6.1 Erweiterter Netzwerk- und Performance-Test

Um noch detailliertere Einblicke in den Datenverkehr und mögliche Fehlerquellen zu erhalten, haben wir einen erweiterten Netzwerk- und Performance-Test mit unserem Skript `tcpdump-analyse.sh` durchgeführt. Dieses Skript ermöglicht es, die Laufzeit der Analyse flexibel festzulegen. Für diesen Test haben wir eine Laufzeit von 250 Sekunden gewählt.

In dieser Zeit wurden insgesamt 355.313 Pakete erfasst. Alle Pakete wurden erfolgreich verarbeitet, und es gab keine vom Kernel verworfenen Pakete, was auf eine saubere Erfassung und stabile Netzwerkbedingungen hinweist. Bei der detaillierten Analyse der erfassten Daten zeigte sich, dass alle Pakete HTTP-Anfragen über Port 80 betrafen. Insgesamt wurden 355.313 HTTP-Anfragen registriert. Auffällig war, dass es während der gesamten Laufzeit keine MariaDB- oder Redis-Operationen gab. Dies deutet darauf hin, dass diese Dienste während der Analysezeit entweder nicht aktiv genutzt wurden oder dass der entsprechende Traffic außerhalb der erfassten Ports stattfand.

Die HTTP-Statuscode-Analyse zeigte, dass von den 118.366 erfolgreich abgeschlossenen HTTP-Anfragen fast alle den Statuscode 200 (OK) erhielten. Lediglich eine einzige Anfrage erhielt den Statuscode 400 (Bad Request), was möglicherweise auf einen fehlerhaften Request oder eine falsche URL hindeutet. Bei der Fehleranalyse in den Logs wurden insgesamt 2 Fehler und 1 Timeout festgestellt. Obwohl diese Zahlen im Vergleich zur Gesamtzahl der Anfragen minimal erscheinen, deuten sie darauf hin, dass gelegentliche Verzögerungen oder kleinere Probleme

auftreten können. Diese könnten auf temporäre Netzwerkprobleme, einen kurzen Engpass bei der Serverlast oder einen fehlerhaften Client-Request zurückzuführen sein.

7.7 k6 Lasttest – Performance-Analyse von HAProxy und Apache

Um die Performance und Stabilität unserer HAProxy- und Apache-Serverinstanzen zu testen, haben wir einen umfassenden Lasttest mit k6 durchgeführt. Das Hauptziel war es, die Antwortzeiten und die Erfolgsquote der Anfragen zu analysieren sowie potenzielle Engpässe zu identifizieren.

Insgesamt wurden im Testlauf 889.788 HTTP-Anfragen durchgeführt, die sich auf die Dienste HAProxy, Apache1 und Apache2 verteilten. Alle drei Systeme arbeiteten zuverlässig, da 99 % der Anfragen erfolgreich bearbeitet wurden. Die Erfolgsquote war damit nahezu fehlerfrei.

Die Analyse der Antwortzeiten zeigte, dass die Mehrheit der Anfragen innerhalb der akzeptablen Grenze von 500 ms blieb. Allerdings gab es einige langsame Anfragen: Bei HAProxy wurden 72 Anfragen, bei Apache1 92 Anfragen und bei Apache2 96 Anfragen registriert, die die Zielzeit von 500 ms überschritten. Dies entspricht jedoch weniger als 1 % der Gesamtanfragen, was auf ein insgesamt stabiles und performantes System hinweist.

Die durchschnittliche Antwortzeit betrug 10,43 ms, wobei die langsamsten Anfragen bis zu 2,99 Sekunden benötigten. Die 90 % der schnellsten Anfragen lagen unter 21,87 ms, während 95 % der Anfragen unter 28,68 ms abgeschlossen wurden. Dies zeigt, dass die überwiegende Mehrheit der Anfragen schnell verarbeitet wurde und nur vereinzelt längere Wartezeiten auftraten.

Insgesamt wurden während des Tests 1,7 GB Daten empfangen und 68 MB Daten gesendet, was einem durchschnittlichen Datendurchsatz von 11 MB/s beim Empfang und 437 kB/s beim Senden entspricht. Die hohe Anzahl an erfolgreichen Anfragen und der stabile Datendurchsatz sprechen dafür, dass unser System auch bei hoher Last effizient arbeitet.

Die durchgeführten Tests zeigen, dass unsere Infrastruktur insgesamt stabil und leistungsfähig ist. Die hohe Anzahl an erfolgreich verarbeiteten Anfragen und der geringe Anteil an Fehlern sprechen für eine gut konfigurierte Umgebung. Die wenigen Fehler und Timeouts deuten darauf hin, dass unter sehr hoher Last gelegentlich kleine Verzögerungen auftreten können. Hier könnten Optimierungen an den Timeouts in HAProxy, MariaDB oder Redis vorgenommen werden, um das System noch robuster zu machen.

Zusätzlich könnte es sinnvoll sein, in zukünftigen Tests den Traffic von MariaDB und Redis gezielt zu simulieren, um deren Verhalten unter Last genauer zu untersuchen. Auch eine längere Testdauer könnte helfen, sporadisch auftretende Fehler besser zu identifizieren.

Insgesamt sind wir mit den Ergebnissen sehr zufrieden, da unser System die umfangreiche Belastung gut bewältigt hat und zuverlässig auf alle Anfragen reagierte. Die Tests haben uns

wertvolle Einblicke geliefert, und wir sind zuversichtlich, dass wir unsere Infrastruktur weiter optimieren können, um zukünftige Lastspitzen noch besser zu bewältigen.

7.7.1 k6 Login-Lasttest

Um die Stabilität und Performance des Login-Prozesses zu überprüfen, haben wir einen umfangreichen Lasttest mit k6 durchgeführt. Ziel war es, sicherzustellen, dass Benutzer sich erfolgreich anmelden können und dass das Session-Cookie korrekt gesetzt wird. Der Test lief über einen Zeitraum von 2 Minuten mit maximal 100 parallelen virtuellen Nutzern (VUs).

Während des Tests wurden insgesamt 7.506 HTTP-Anfragen gestellt, wobei 100 % der Anfragen erfolgreich waren. Es gab keine technischen Fehler oder Abbrüche, was zeigt, dass der Login-Prozess stabil und zuverlässig funktioniert. Bei allen 3.753 Login-Versuchen wurde der Login erfolgreich abgeschlossen, und das Session-Cookie wurde korrekt gesetzt. Dies bestätigt, dass die Authentifizierung sauber umgesetzt ist und einwandfrei funktioniert. Zudem konnte nach dem Login die ToDo-Seite ohne Probleme geladen werden, was die korrekte Funktionsweise der Session-Verwaltung unterstreicht.

Die detaillierte Analyse der Performance-Daten zeigt, dass insgesamt 29 MB an Daten empfangen und 1,6 MB an Daten gesendet wurden. Der durchschnittliche Datendurchsatz lag bei 216 kB/s beim Empfang und 12 kB/s beim Senden. Die durchschnittliche Antwortzeit betrug 443,65 ms, was für eine größere Anzahl von gleichzeitigen Nutzern noch akzeptabel ist. Allerdings gab es einige längere Wartezeiten, wobei die maximale Antwortzeit bei 4,47 Sekunden lag. Die Statistik zeigt, dass 90 % der Anfragen schneller als 1,02 Sekunden und 95 % der Anfragen schneller als 1,24 Sekunden bearbeitet wurden. Der Test zeigt, dass der Login-Prozess zuverlässig funktioniert und es keine technischen Fehler oder abgebrochenen Anfragen gab. Dies ist ein wichtiger Erfolg, da die Kernfunktionen des Systems stabil und fehlerfrei arbeiten. Allerdings deuten die längeren Antwortzeiten darauf hin, dass unter hoher Last gelegentlich Verzögerungen auftreten können. Hier besteht noch Optimierungspotenzial, um die Performance weiter zu verbessern.

Mögliche Maßnahmen zur Verbesserung könnten die Optimierung der Datenbankabfragen sein, wodurch die Antwortzeiten reduziert werden könnten. Durch effizientere Abfragen ließe sich die Belastung auf die Datenbank verringern, was zu schnelleren Antwortzeiten führen würde. Eine weitere Möglichkeit ist das Caching von Inhalten. Häufig abgerufene Daten könnten zwischengespeichert werden, um die Last auf dem Server zu verringern und die Zugriffszeiten zu verkürzen. Zusätzlich könnte die Skalierung der Serverressourcen helfen, Engpässe zu vermeiden. Eine bessere Lastverteilung oder zusätzliche Ressourcen könnten die Leistung des Systems unter hoher Belastung verbessern.

Insgesamt zeigt der Test, dass der Login-Prozess stabil arbeitet und alle Authentifizierungen korrekt ablaufen. Die grundlegenden Funktionen, wie das Laden der ToDo-Seite nach dem Login, funktionieren zuverlässig. Trotz der leicht erhöhten Wartezeiten in einigen Fällen können

wir den aktuellen Stand als erfolgreich betrachten. Um die Leistung weiter zu verbessern, sollten wir gezielte Optimierungen an den Antwortzeiten und der Skalierbarkeit des Systems vornehmen.

7.7.2 k6 Add-ToDo-Liste

Um die Stabilität und Zuverlässigkeit unserer ToDo-Listen-Funktion unter Belastung zu testen, haben wir einen umfassenden Lasttest mit dem Tool k6 durchgeführt. Ziel war es, herauszufinden, wie die Anwendung bei steigender Nutzerzahl reagiert und ob es zu Verzögerungen oder Fehlern kommt. Der Test lief insgesamt 1 Minute und 27 Sekunden und war in drei Abschnitte unterteilt: Zuerst simulierten wir 20 gleichzeitige Nutzer für 20 Sekunden, um eine moderate Last zu simulieren. Danach erhöhten wir die Anzahl der Nutzer auf 50 für 40 Sekunden, um das System unter höherer Belastung zu testen. Schließlich reduzierten wir die Anzahl der virtuellen Nutzer wieder auf 0, um zu prüfen, wie das System mit abnehmender Last umgeht.

Während des Tests wurden insgesamt 2.007 vollständige Abläufe durchgeführt, bei denen 6.021 HTTP-Anfragen an das System gesendet wurden. Besonders positiv ist, dass 100 % der Aufgaben erfolgreich hinzugefügt wurden und die Anzeige der ToDo-Liste einwandfrei funktioniert hat. Dies zeigt, dass unsere Anwendung stabil und zuverlässig arbeitet.

In Bezug auf die übertragenen Daten wurden insgesamt 4,5 MB empfangen und 1,1 MB gesendet. Dies entspricht einem Datendurchsatz von ca. 51 kB/s beim Empfang und 12 kB/s beim Senden.

Die gemessenen Antwortzeiten verdeutlichen, dass unser System auch unter Belastung effizient arbeitet. Die durchschnittliche Antwortzeit betrug lediglich 17,57 ms, was für eine Webanwendung sehr schnell ist. Die kürzeste Antwortzeit lag bei beeindruckenden 357 µs (Mikrosekunden), und selbst die langsamsten Anfragen wurden noch innerhalb von 159 ms verarbeitet, was ebenfalls ein guter Wert ist. Die Statistik zeigt außerdem, dass 90 % der Anfragen schneller als 48,3 ms und 95 % der Anfragen schneller als 55,04 ms abgeschlossen wurden.

Besonders erfreulich ist, dass es während des gesamten Tests keine Fehler gab. Alle Anfragen wurden erfolgreich bearbeitet, und es traten keine technischen Probleme auf. Dies spricht dafür, dass unser System robust und zuverlässig arbeitet und auch bei hoher Belastung keine Ausfälle zeigt. Die Skalierung der Nutzerzahlen verlief ebenfalls reibungslos, was zeigt, dass unser System flexibel auf unterschiedliche Lasten reagieren kann.

Was bedeutet das für uns?

Die Tests zeigen, dass die ToDo-Listen-Funktion derzeit sehr stabil und leistungsfähig ist. Die Anwendung verarbeitet Anfragen zuverlässig und schnell, selbst wenn viele Nutzer gleichzeitig aktiv sind. Dass 100 % der Aufgaben erfolgreich hinzugefügt wurden und es keine Fehler gab, zeigt, dass das System technisch sauber arbeitet.

Trotz der sehr positiven Ergebnisse gibt es kleinere Optimierungsmöglichkeiten. Die maximale Antwortzeit von 159 ms könnte durch weitere Verbesserungen reduziert werden. Mögliche Maßnahmen hierfür sind die Optimierung der Datenbankabfragen, um effizientere Abfragen zu ermöglichen, das gezielte Caching häufig abgerufener Daten, um die Last auf dem Server zu verringern, und eine verbesserte Netzwerk-Konfiguration, um Engpässe zu vermeiden.

Fazit

Insgesamt zeigt der Test, dass unsere ToDo-Listen-Funktion stabil und zuverlässig arbeitet. Die Anwendung bewältigt auch unter hoher Last eine große Anzahl von Anfragen ohne Fehler. Mit einigen gezielten Optimierungen könnten wir die Performance weiter verbessern und die Antwortzeiten noch weiter reduzieren. Die Ergebnisse bestätigen, dass unser System technisch gut aufgestellt ist und eine solide Grundlage für zukünftige Erweiterungen bietet.

7.8 Diagramm – Gnuplot: CPU- und RAM-Nutzungsanalyse

Im Rahmen unseres Performance-Monitoring-Tests haben wir ein Skript (`monitor.sh`) verwendet, das kontinuierlich CPU- und RAM-Nutzungsdaten von mehreren Docker-Containern (Apache1, Apache2, Work, HAProxy, Redis und MariaDB) erfasst hat. Die Daten wurden in Intervallen von 1 Sekunde über einen längeren Zeitraum gesammelt und in einer CSV-Datei gespeichert, um sie anschließend zu analysieren und zu visualisieren.

Testaufbau

Das Skript `monitor.sh` nutzte den Befehl `docker stats`, um kontinuierlich Daten zu sammeln. Die erfassten Daten umfassten die CPU-Auslastung (%) und den RAM-Verbrauch (MiB) für die Container Apache1, Apache2, Work, HAProxy, Redis und MariaDB. Die Daten wurden in Intervallen von 1 Sekunde erfasst, um detaillierte Schwankungen und Lastspitzen zu erkennen. Die gesammelten Daten wurden in einer CSV-Datei gespeichert und anschließend mithilfe von Gnuplot visualisiert. Das Diagramm zeigt die CPU-Auslastung der verschiedenen Docker-Container zwischen 19:00 Uhr und 21:00 Uhr und gibt uns einen guten Überblick darüber, wie das System unter Belastung reagiert hat.

Allgemeines CPU-Verhalten

Die meisten Container liefen stabil, mit gelegentlichen Spitzen, die bei hoher Last zu erwarten sind. Die CPU-Nutzung blieb meist unter 40 %, was auf eine effiziente Ressourcennutzung hindeutet. Allerdings gab es einige Ausreißer, insbesondere bei den Containern Work, HAProxy und MariaDB.

Auffällige Spitzen in der CPU-Nutzung

Die Container Work (grüne Linie) und HAProxy (rote Linie) zeigten teilweise Spitzen von über 100 %. Dies deutet darauf hin, dass diese Container bei starker Belastung, z. B. durch viele gleichzeitige Anfragen oder aufwendige Berechnungen, stark beansprucht wurden. Die Spitzen

bei Work korrelieren mit unserem wrk-Test, der gezielt solche Stresssituationen simulierte. HAProxy hatte ebenfalls hohe Lastspitzen, was darauf hindeutet, dass das Load-Balancing zeitweise stark gefordert war.

Stabile Container

Die Container Apache1 und Apache2 zeigten ein entspanntes Verhalten mit nur gelegentlichen, kleineren Ausreißern. Dies zeigt, dass die Webserver stabil liefen und ihre Aufgaben gut verteilt waren. Redis blieb durchgehend ruhig, was darauf hindeutet, dass der In-Memory-Cache effizient arbeitete. MariaDB hatte meist niedrige Werte, zeigte aber gelegentlich Spitzen bis zu 60%. Diese Spitzen könnten auf größere Datenbankabfragen oder komplexe Operationen zurückzuführen sein.

Schwankungen und Beruhigung

Gegen Ende des Tests normalisierten sich die CPU-Werte wieder. Dies zeigt, dass sich das System nach der hohen Belastung gut erholte und wieder stabil lief.

Was läuft gut und was könnte besser werden?

Das System zeigte insgesamt ein stabiles Verhalten. Die CPU-Auslastung blieb meist im grünen Bereich, was auf eine effiziente Ressourcennutzung hindeutet. Die CPU-Spitzen bei Work und HAProxy traten vor allem während gezielter Belastungstests auf, was die Belastbarkeit des Systems unterstreicht.

Dennoch gibt es Optimierungspotenzial. Die starken CPU-Spitzen, insbesondere bei Work, könnten durch effizienteren Code, besseres Thread-Management oder eine gezieltere Verteilung der Ressourcen verringert werden. Bei HAProxy könnten feinere Einstellungen beim Load-Balancing und klügere Timeout-Regeln die Belastung auf den Container reduzieren. Für MariaDB könnten Maßnahmen wie bessere Indizes, Query-Caching oder parallele Abfragen helfen, Lastspitzen zu vermeiden.

Fazit

Die Analyse der CPU- und RAM-Nutzung zeigt, dass unser System insgesamt stabil und effizient arbeitet. Die meisten Container bewältigten die Belastung gut, und die CPU-Spitzen traten vor allem während gezielter Stresssituationen auf. Dennoch gibt es Optimierungspotenzial, insbesondere bei der Reduzierung von Lastspitzen und der Feinjustierung der Ressourcenverteilung. Mit gezielten Anpassungen können wir die Performance weiter verbessern und das System noch robuster machen.

8 Fazit und Ausblick

Literaturverzeichnis

- [1] D. E. Knuth, *The art of computer programming / Donald E. Knuth: Combinatorial algorithms Fasc. 1: Bitwise tricks & techniques : binary decision diagrams*. Upper Saddle River, NJ [u.a.]: Addison-Wesley, 2009, ISBN: 0321580508.
- [2] F. Lignitz. „How-to LaTeX.“ de, besucht am 18. Apr. 2023. Adresse: <https://informatik.hs-bremerhaven.de/flignitz/latex.html>.

Abbildungsverzeichnis

3.1	Verzeichnisstruktur	6
4.1	Deckblatt	11
5.1	Beispiel für eine section	14
5.2	Beispiel für eine subsection	15
6.1	BibTeX Einträge exportieren	27
6.2	BibTeX Einträge exportieren (Formatauswahl)	28
I.1	Poster	48
II.1	Frame mit Listing	51

Tabellenverzeichnis

5.1	L ^A T _E X Schriftgrößen	15
5.2	L ^A T _E X Sonderzeichen	16
5.3	float specifier	18
5.4	Tabelle mit zwei Spalten	23
5.5	Tabelle mit drei Spalten	24
5.6	Tabelle mit drei Spalten und angepasster Spaltenbreite	25

Listingverzeichnis

4.1	Die documentclass	7
4.2	Package Definition	8
4.3	Auswahl der Zitierstile	9
4.4	Definition des Deckblatts	10
4.5	Kopf- und Fußzeile	12
4.6	input statements	12
5.1	Textbeispiel	14
5.2	Textbeispiel 2	14
5.3	Textbeispiel 3	16
5.4	Floatumgebung für figures	18
5.5	Beispiel FloatBarrier	19
5.6	Beispiel 2 FloatBarrier	19
5.7	Beispiel für das Einbinden einer Abbildung	20
5.8	Beispiel 2 für das Einbinden einer Abbildung	20
5.9	Beispiel für ein Listing	21
5.10	Beispiel 2 für ein Listing	21
5.11	Inline Listings	21
5.12	Beispielcode für Tabelle	22
5.13	Beispielcode für Tabelle mit drei Spalten	24
5.14	Beispielcode für Tabelle mit drei Spalten und angepasster Spaltenbreite	25
5.15	Beispiel für eine longtblr	26
5.16	Beispiel für ein Label	26
5.17	Beispiel für eine Referenz	27
6.1	Beispiel BibTeX Eintrag	28
6.2	Beispiel BibTeX Eintrag (Onlinequelle)	29
6.3	Beispiele zitieren	30
II.1	Beamer Frame	49
II.2	Beamer Frame mit Listing	50
III.1	Einbinden von subfiles	52
III.2	Erstellen von subfiles	52

Anhang

I Die Poster

Passend zu diesem Template existieren noch zwei weitere Vorlagen für Poster. Diese unterscheiden sich lediglich durch die Ausrichtung der Seite und sind ansonsten identisch. Der Aufbau der Dateien und Projektstruktur ist weitestgehend identisch zu dieser Vorlage, es gilt lediglich auf einige kleine Besonderheiten zu verweisen.

In der sog. multicols Umgebung ist es nicht möglich, mit Floatumgebungen zu arbeiten, dies ist für fast alle Beispiele aus dieser Anleitung irrelevant, lediglich Grafiken müssen anders eingebunden werden. Dazu finden sich jedoch Beispiele in den Vorlagen. Ebenso ist ein Teil der Präambel in eine separate Datei ausgelagert, in dieser sollten auch nicht ohne weiteres Änderungen vorgenommen werden. Wie in diesem Template, müssen innerhalb der Präambel Zitierstil, sowie Autor:innen und Titel gesetzt werden.

Die Vorlagen können auf meiner Webseite sowohl angeschaut, als auch heruntergeladen werden, alle Hinweise zum Kompilieren gelten für die Poster ebenso. Ergebnis dieser Vorlage ist z. B. folgendes Poster:

Eine unnötig komplizierte und sehr sehr lange Abhandlung

Untersuchung einer Sache von wirklichem Wert

Maxi Mustermensch

Einleitung

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Zielstellung

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Befehl	Ergebnis
\tiny	klein
\scriptsize	klein
\footnotesize	etwas größer
\small	noch etwas größer
\normalsize	normal
\large	groß
\LARGE	größer
\huge	noch größer
\Huge	riesig
\Huge	noch riesiger

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Materialien und noch etwas, damit es zweizeilig wird

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Hochschule Bremerhaven

sie eine falsche Anmutung vermitteln.

```
for i in {1..10}; do
  echo "81"
done
```

Listing 1: Ein Listing

Schluss

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.



Abbildung 1: Bildunterschrift [1, S.14]

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Haardest gefbarn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Referenzen

- [1] M. M. Mustermensch „MS Windows NT Kernel Description.“ de. (Dez. 2012). Adresse: <http://www.800militaria.com/wins/ntkernel.html> (besucht am 30.09.2012).

Abbildung I.1: Poster

II Beamer

$\text{\LaTeX}$ bietet nicht nur die Möglichkeit schriftliche Ausarbeitungen in Form von Hausarbeiten zu erstellen, sondern mit Beamer eine Alternative zu PowerPoint oder Keynote. Das Layout ist in vielen Fällen schlicht, was nicht negativ gemeint ist und erinnert im Design eher an die frühen 2000er. Durch einige Anpassungen ist es dennoch möglich ein schlichtes, aber funktionales Layout zu entwerfen, welches sich gut für Präsentationen eignet.

Eine solche Vorlage findet sich analog zu den Poster Vorlagen und erfordert wenig Umgewöhnung. Es können Tabellen und Abbildungen, sowie Referenzen wie gewohnt genutzt werden. Lediglich die Listings erfordern eine kleine Umgewöhnung, ebenso wie das Arbeiten mit der *column* Umgebung.

```

1 \section{Frame Example with table}
2
3 \begin{frame}{Frame Example with table}
4   \begin{columns}
5
6     \begin{column}{0.4\textwidth}
7       This is some text in the second frame~\autocite{donovan2015go}.
8       This is some text in the second~\autocite{kane2018docker}.
9     \end{column}
10
11    \begin{column}{0.59\textwidth}
12      \scriptsize{
13        \begin{talltblr}[caption={\LaTeX~Sonderzeichen}]{colspec={X[1,m] X[0.5,c,m]},
14          ↪ rowhead=1, width=0.8\textwidth}\toprule
15        Befehl                                & Ergebnis                                & \midrule
16        \textbackslash\&                      & \&                                    & \cmidrule{1-2}
17        \textbackslash\%                      & \%                                    & \cmidrule{1-2}
18        \textbackslash\$                      & \$                                    & \cmidrule{1-2}
19        \textbackslash\#                      & \#                                    & \cmidrule{1-2}
20        \textbackslash\{                      & \{                                    & \cmidrule{1-2}
21        \textbackslash\}                      & \}                                    & \cmidrule{1-2}
22      \end{talltblr}}
23    \end{column}
24  \end{columns}
25 \end{frame}

```

Listing II.1: Beamer Frame

Wie zu sehen, wird in Beamer ebenfalls mit sections gearbeitet. Um einen neuen Slide zu erstellen wird `\begin{frame}` genutzt. Die `columns` Umgebung ermöglicht das Unterteilen des Frames in mehrere Spalten, welche daraufhin als einzelne `column` definiert werden. Dabei kann die Textbreite angegeben werden.

Listings müssen über eine besondere Umgebung definiert und eingebunden werden, da Beamer ansonsten nicht mit `minted` umgehen kann.

```
1 \defverbatim[colored]\CodeOne{
2   \begin{minted}{python}
3     import numpy as np
4     import pylab as pl
5
6     def f_x(x):
7       return np.exp(x)+x**2-5*x
8
9     def approx_f(x):
10      return 1 -4*x +3./2*x**2
11
12     xvals = np.arange(-4,4,0.1)
13     fx_vals = [f_x(x) for x in xvals]
14     approx_vals = [approx_f(x) for x in xvals]
15
16     pl.plot(xvals,fx_vals)
17     pl.plot(xvals,approx_vals)
18
19     pl.show()
20 \end{minted}
21 \captionof{listing}{Example Code}
22 }
23
24 \section{Code Example}
25 \begin{frame}{Code Example}
26   Some Text on first Frame~\autocite{gregg2013systems}
27   \CodeOne
28   \vspace{0.5cm}
29 \end{frame}
```

Listing II.2: Beamer Frame mit Listing

Wie im Listing gezeigt, wird das Listing außerhalb des Frames definiert und in Zeile 27 eingefügt. CodeOne ist dabei der zugewiesene Name unter dem das Listing eingefügt werden kann. Die Definition erfolgt in Zeile 1, der Name ist frei wählbar. Das Listing so einzubinden ist nicht optimal, jedoch bleibt aktuell keine andere Möglichkeit, wenn mit `minted` gearbeitet wird. Das Inhaltsverzeichnis und die Referenzen werden automatisch erstellt, alle Einstellungen können natürlich innerhalb der Vorlage angepasst werden. Obiges Listing würde mit der Vorlage folgendermaßen aussehen. Wie immer finden sich alle Beispiele auch in den jeweiligen Vorlagen.

Code Example

Some Text on first Frame [1]

```

1  import numpy as np
2  import pylab as pl
3
4  def f_x(x):
5      return np.exp(x)+x**2-5*x
6
7  def approx_f(x):
8      return 1 -4*x +3./2*x**2
9
10 xvals = np.arange(-4,4,0.1)
11 fx_vals = [f_x(x) for x in xvals]
12 approx_vals = [approx_f(x) for x in xvals]
13
14 pl.plot(xvals,fx_vals)
15 pl.plot(xvals,approx_vals)
16
17 pl.show()

```

Listing: Example Code

**Hochschule
Bremerhaven**

Maxi Musterfrau (12345), Maxi Mustermann (54321)
Titel
10. August 2021
4 / 12

Abbildung II.1: Frame mit Listing

III Quickbuild

Wird ein Projekt mit der Zeit relativ groß, verfügt also über eine Vielzahl von Listings, Tabellen, Abbildungen und Text, dann wird für das Kompilieren eine nicht unerhebliche Zeit benötigt. Um das ganze Dokument, inkl. aller Referenzen und Abkürzungen etc. zu bauen, ist dies auch nötig und lässt sich nicht vermeiden. Oft benötigt man jedoch nur einen kurzen Blick auf Layout oder den Umfang des bisher geschriebenen und möchte nicht das Dokument in seiner Gesamtheit kompilieren.

Für diese Anforderungen gibt es sog. subfiles, diese ermöglichen es, nur einen kleinen Teil des Dokumentes zu kompilieren und als PDF zu betrachten. Dabei kommt es zu gewissen Einschränkungen, so werden Seitenzahlen, Referenzen und Abkürzungen nicht korrekt dargestellt, dennoch erhält man einen guten Blick auf das Layout. Um mit subfiles zu arbeiten, muss einiges beachtet werden. So muss statt des `\input{}` das Kommando `\subfile{}` genutzt werden. Darüber hinaus müssen alle `.tex`-Dateien mit einer eigenen `documentclass` ausgestattet werden. In der hauptdatei `.tex` sieht dies dann folgendermaßen aus.

```
1 \subfile{src/einleitung.tex}
2 \subfile{src/struktur.tex}
3 \subfile{src/zurhauptdatei.tex}
```

Listing III.1: Einbinden von subfiles

Jedes Subfile muss dann mit der passenden `documentclass` beginnen.

```
1 \documentclass[../hauptdatei.tex]{subfiles}
2 \begin{document}
3 \section{Beispiel}
4
5 Hier könnte sinnvoller Inhalt stehen.
6
7 \end{document}
```

Listing III.2: Erstellen von subfiles

Durch die eigene `documentclass`, welche wiederum auf die `hauptdatei.tex` verweist und die Angabe von `\begin` bzw. `\end{document}` kann die entsprechende `tex`-Datei separat kompiliert werden. Dafür kann das Skript `quickbuild.sh` genutzt werden, welches als Argument den Namen der zu kompilierenden Datei erwartet und ein PDF erzeugt.

Abbildungen, Listings, Tabellen können wie gewohnt genutzt werden. Wird `buildlualatex.sh` ausgeführt, baut nach wie vor das gesamte Dokument inkl. aller Subfiles und Referenzen.

Selbstständigkeitserklärung

Ich versichere, die von mir vorgelegte Arbeit selbstständig verfasst zu haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben. Die Arbeit habe ich mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegt.

Bremerhaven, den 9. März 2025

Unterschrift: